



DeepL

Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

Ознайомлення з технологією «Генерація з розширеним пошуком»

Генерація з розширеним пошуком (RAG) — це техніка, яка допомагає працювати з великими документами, що занадто великі, щоб вміститися в один запит. Замість того, щоб втискувати все в один величезний запит, RAG розбиває документи на частини та включає лише найрелевантніші фрагменти під час відповіді на запитання.

Проблема з великими документами

Уявіть, що у вас є фінансовий документ на 800 сторінок, і ви хочете задати Claude конкретні запитання щодо нього, наприклад: «Які фактори ризику має ця компанія?» Вам потрібно якимось чином передати Claude релевантну інформацію з документа, але існують обмеження щодо обсягу тексту, який можна включити в запит.

Claude with large documents

- We might want to ask Claude questions about specific sections of a large document
 - “What risk factors does this company have?”
- It's a private document – Claude won't have any idea what is contained in it
- How can we get Claude to understand the contents?

ANTHROPIC

Financial Doc, 800+ pages

1. Executive Summary

Corporation ("the Company") is pleased to present its Annual Financial Report for the fiscal year ended December 31, 2024. The Company has demonstrated strong financial performance across all key metrics, with revenue growth of 12.3% year-over-year, reaching \$1.24 billion. Operating income increased by 15.7% to \$342 million, while net income rose by 18.2% to \$267 million.

The Board of Directors has approved a quarterly dividend of \$0.45 per share, representing a 10% increase from the previous quarter. This dividend will be payable on March 15, 2025, to shareholders of record as of February 28, 2025.

The Company continues to invest in research and development, with R&D expenses increasing by 22.5% to \$156 million. These investments are focused on expanding our product portfolio and enhancing our technological capabilities to maintain our competitive edge in the market.

2. Management Discussion & Analysis

Overview of Business Operations

Corporation operates in three primary business segments: Consumer Products, Enterprise Solutions, and Digital Services. The Consumer Products segment, which accounts for 45% of total revenue, experienced growth of 9.8% year-over-year. The Enterprise Solutions segment, representing 35% of total revenue, grew by 14.2%. The Digital Services segment, which accounts for 20% of total revenue, demonstrated the strongest growth at 18.7%.

Market Conditions and Competitive Landscape

The global market for our products and services remained competitive throughout the fiscal year. We observed increased competition in the Consumer Products segment, particularly in emerging markets. However, our strategic focus on premium offerings and brand differentiation allowed us to maintain our market share and improve profit margins.

Risk Factors

The Company continues to monitor several risk factors that could impact future performance, including:

- Fluctuations in raw material costs and supply chain disruptions
- Changes in regulatory requirements across our operating regions
- Technological disruptions and cybersecurity threats
- Economic uncertainty in key markets
- Foreign exchange rate volatility

Future Outlook

Looking ahead to fiscal year 2025, we anticipate continued growth across all business segments. We expect revenue growth of 10-12%, driven by new product launches and expansion into new geographic markets. We plan to maintain our focus on operational efficiency,

Варіант 1: Включити все в запит

Перший підхід є простим — витягнути весь текст із документа та вставити його у запит разом із запитанням користувача. Ваш запит може виглядати так:

Дайте відповідь на запитання користувача щодо фінансового документа.

```
<user_question>
{user_question}
</user_question>
```

```
<financial_document>
{financial_document}
</financial_document>
```

Possibly not the best solution

- There's a hard limit on how much text we can feed into Claude - the document might be too long
- Claude gets slightly less effective with very long prompts
- Larger prompts **cost more** to process
- Larger prompts **take longer** to process

ANTHROPIC

Option 1: Include the contents of the document in a prompt

The slide illustrates a prompt structure for Claude. On the left, a snippet of a financial document is shown, titled '1. Executive Summary' and '2. Management Discussion & Analysis'. An arrow points from this document to a dark box on the right, which contains a prompt template. The prompt template starts with 'Answer the user's question about the financial document.' followed by placeholders for the user's question and the financial document content, enclosed in XML-style tags.

```
...
Answer the user's question about
the financial document.

<user_question>
{user_question}
</user_question>

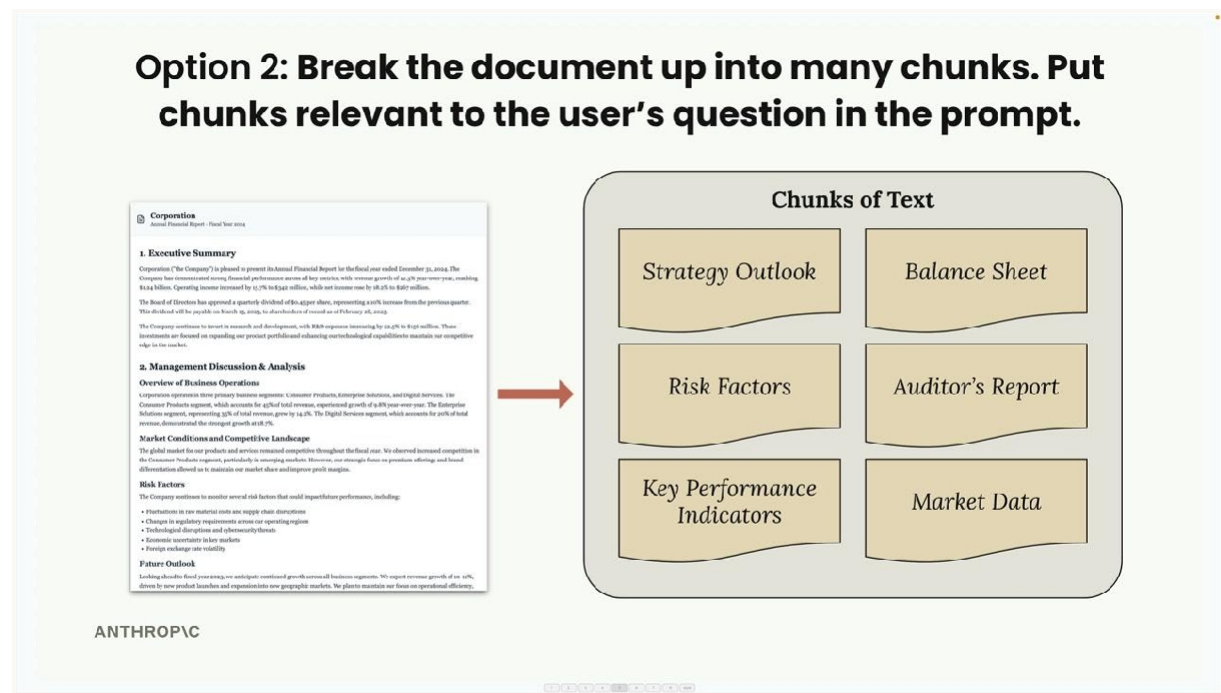
<financial_document>
{financial_document}
</financial_document>
...
```

Цей підхід має серйозні обмеження:

- Існує жорстке обмеження на довжину запиту — ваш документ може виявитися занадто довгим
- Claude стає менш ефективним при дуже довгих підказках
- Обробка більших підказок коштує дорожче
- Обробка великих підказок займає більше часу

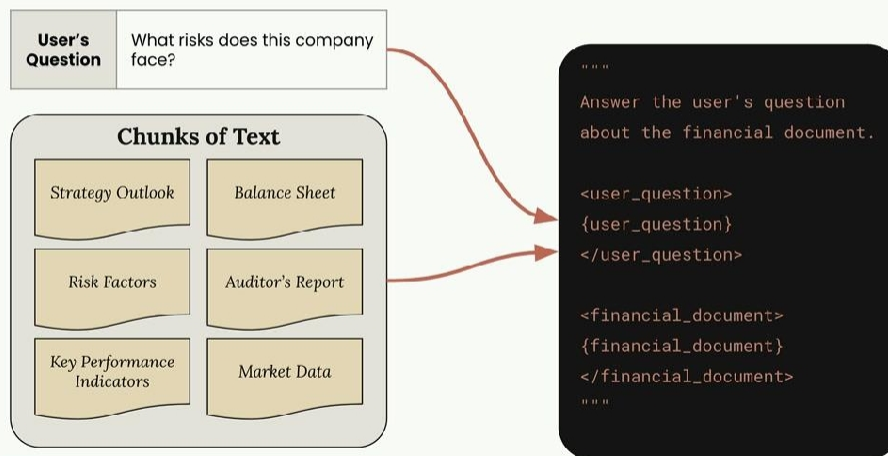
Варіант 2: Розбийте документи на частини

RAG використовує більш розумний підхід. Спочатку під час попередньої обробки ви розбиваєте документ на менші фрагменти. Потім, коли користувач задає питання, ви знаходите фрагменти, які найбільше відповідають його запиту, і включаєте в запит лише їх.



Ось як це працює: якщо хтось запитає «З якими ризиками стикається ця компанія?», ви проскануєте свої фрагменти, знайдете розділ «Фактори ризику» і включите саме цей релевантний фрагмент у свій запит.

Option 2: Break the document up into many chunks. Put chunks relevant to the user's question in the prompt.



ANTHROPIC

Переваги RAG

- Claude може зосередитися лише на найрелевантнішому вмісті
- Підтримує обробку дуже великих документів
- Працює з кількома документами
- Менші запити коштують дешевше та виконуються швидше

Проблеми з RAG

- Потрібна попередня обробка для розбиття документів на фрагменти
- Потрібен механізм пошуку для знаходження «релевантних» фрагментів
- Включені фрагменти можуть не містити всього контексту, необхідного Claude
- Існує багато способів розбиття тексту на фрагменти — який підхід найкращий?

Наприклад, можна розділити документи на частини однакового розміру або створити фрагменти на основі структури документа, такої як заголовки та розділи. Кожен підхід має свої переваги та недоліки, які потрібно оцінити з огляду на конкретний випадок використання.

Коли використовувати RAG

Використання RAG передбачає прийняття багатьох технічних рішень і вимагає більше зусиль, ніж просто включення всього в запит. Вам потрібно проаналізувати, чи переваги переважають складність у вашому конкретному випадку застосування. Це особливо корисно під час роботи з дуже великими документами, декількома документами або коли потрібно оптимізувати витрати та продуктивність.

Головна ідея полягає в тому, що RAG обмінює простоту на масштабованість та ефективність. Хоча для його правильного впровадження потрібно більше початкової роботи, він дає змогу працювати з колекціями документів, з якими неможливо було б впоратися за допомогою простого заповнення промтів.

Стратегії розбиття тексту на фрагменти

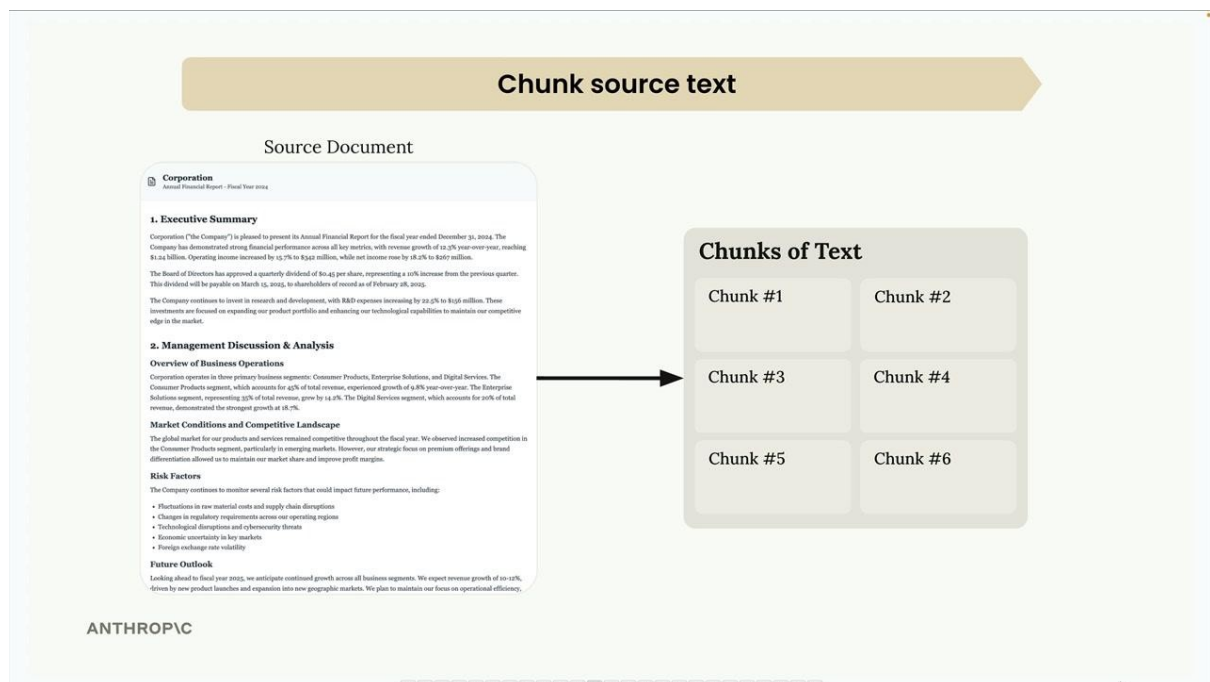
Завантаження

001_chunking.ipynb

(відкривається в новій

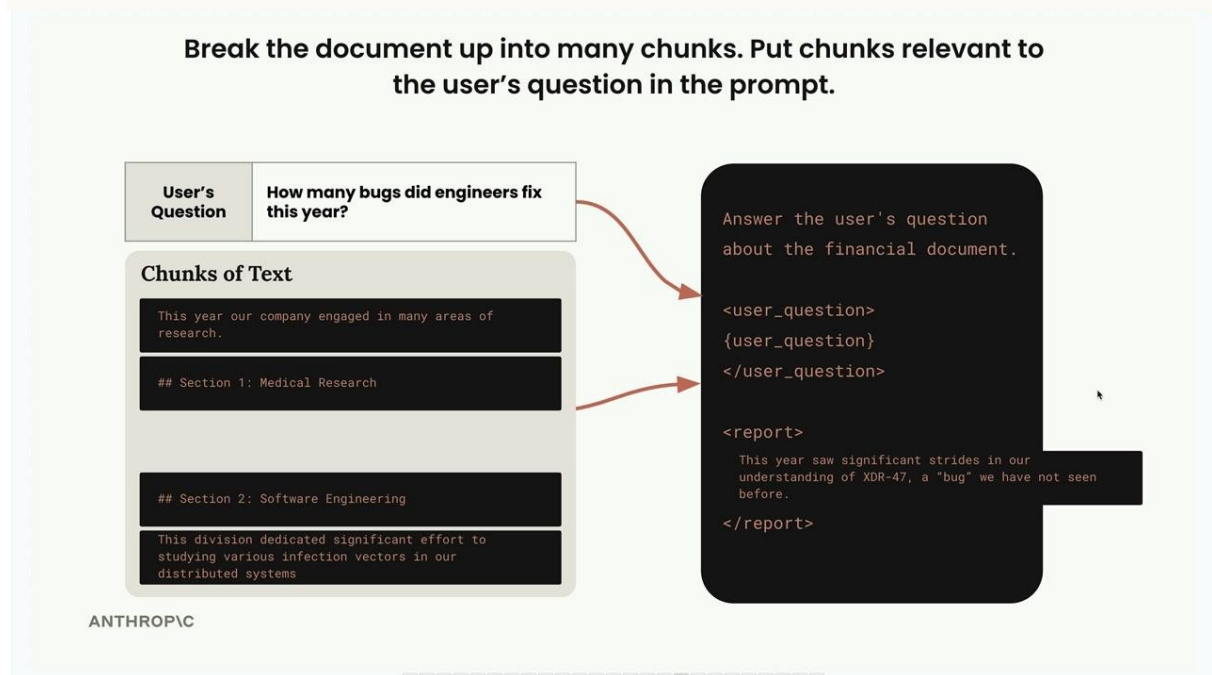
вкладці) report.md

Розбиття тексту на фрагменти є одним із найважливіших етапів побудови конвеєра RAG (Retrieval Augmented Generation). Те, як ви розбиваєте документи, безпосередньо впливає на якість усієї вашої системи. Недосконала стратегія розбиття може призвести до вставки нерелевантного контексту у ваші запити, що змусить ваш ІІІ надавати абсолютно неправильні відповіді.



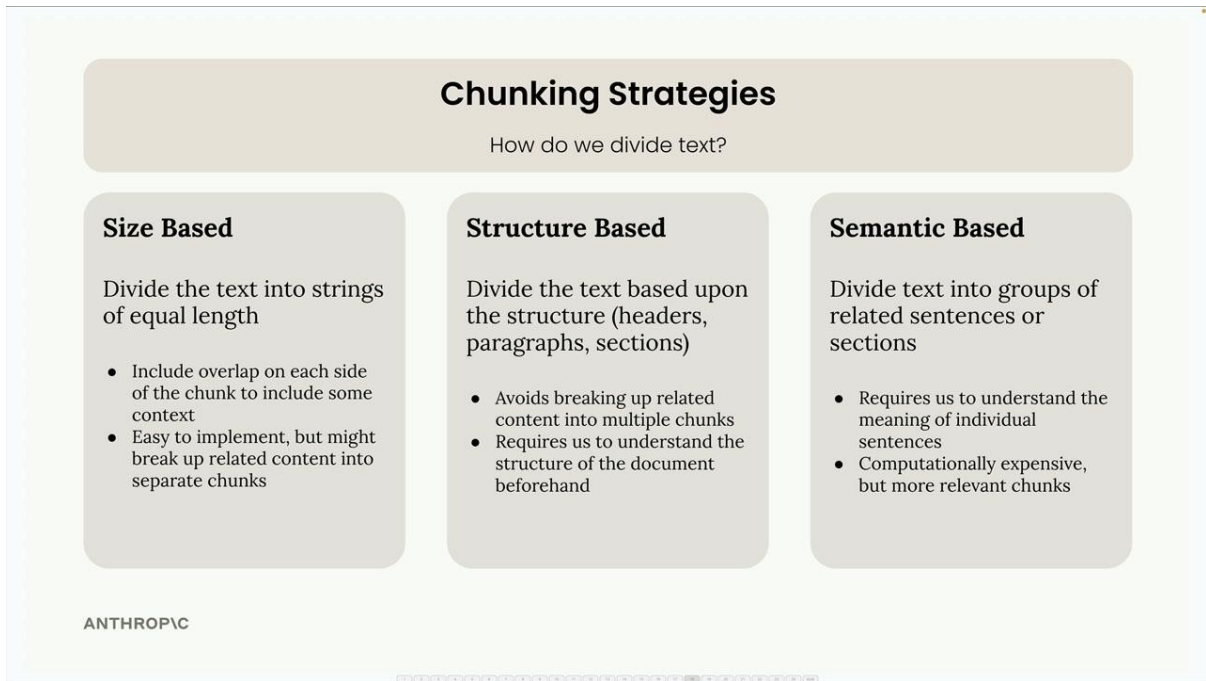
Розгляньмо такий приклад: у вас є документ, що містить розділи про медичні дослідження та розробку програмного забезпечення. Якщо ви неправильно розбиваєте документ на фрагменти, користувач, який запитає: «Скільки помилок виправили інженери цього року?», може отримати інформацію про

медичних досліджень замість розробки програмного забезпечення, просто тому, що в розділі про медицину випадково трапилося слово «bug» в іншому контексті.

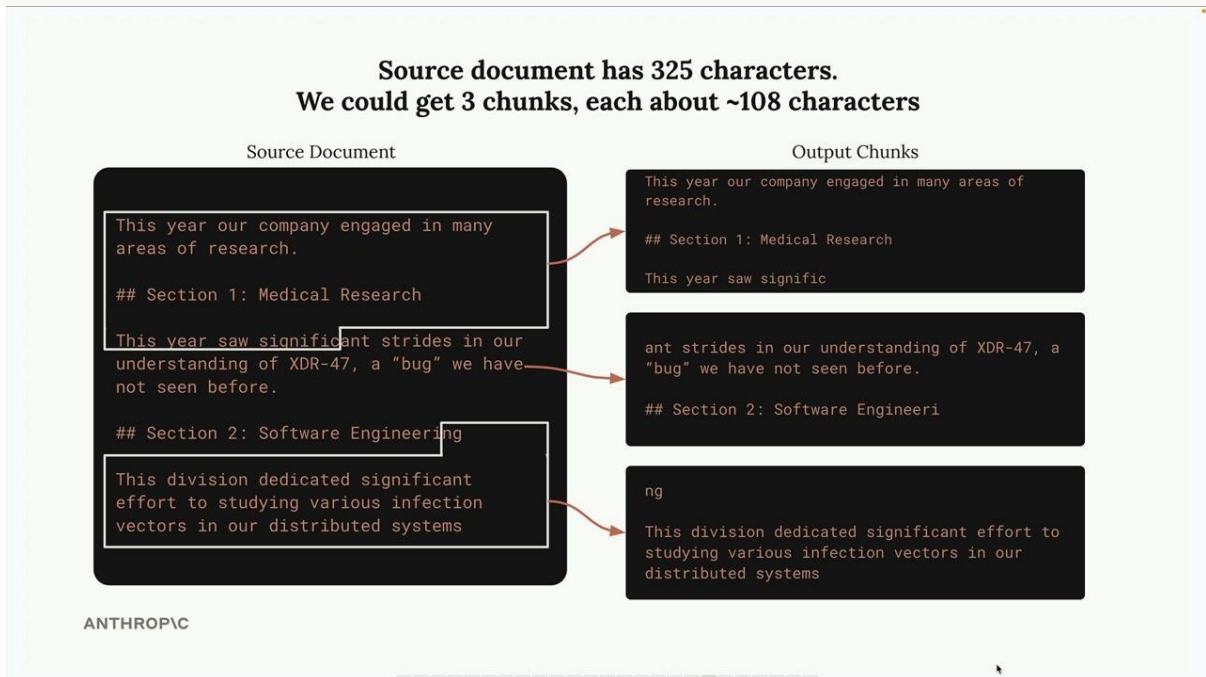


Ось чому вибір правильної стратегії групування має таке велике значення. Давайте розглянемо три основні підходи.

Групування за розміром



Розбиття на фрагменти за розміром — це найпростіший підхід: ви ділите текст на рядки однакової довжини. Якщо у вас є документ із 325 символами, ви можете розділити його на три фрагменти, кожен із яких міститиме приблизно 108 символів.



Цей метод легко реалізувати, і він працює з будь-яким типом документів, але має очевидні недоліки:

- Слова обрізаються посеред речення
- У фрагментах втрачається важливий контекст з навколишнього тексту
- Заголовки розділів можуть бути відокремлені від їхнього змісту

The slide is titled "Sized Based Downsides" and lists two bullet points: "Each chunk has some cutoff text" and "Each chunk lacks context". To the right, three dark rectangular boxes illustrate text truncation. The first box contains "This year our company engaged in many areas of research." and "## Section 1: Medical Research", with the line "This year saw signific" cut off at the bottom. The second box contains "ant strides in our understanding of XDR-47, a 'bug' we have not seen before." and "## Section 2: Software Engineeri", with the line "ng" cut off at the bottom. The third box contains "ng" and "This division dedicated significant effort to studying various infection vectors in our distributed systems", with the line "ng" cut off at the top. The slide footer includes the word "ANTHROPIC" and a navigation bar.

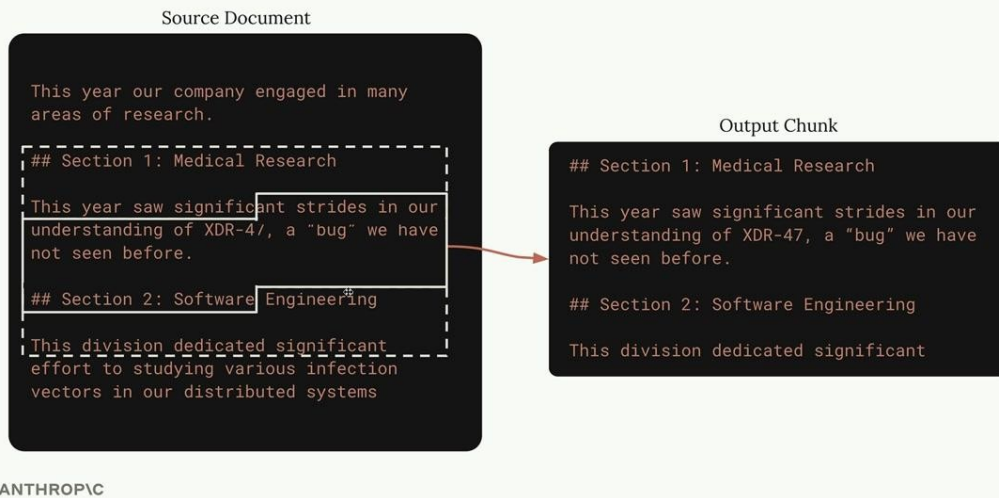
Sized Based Downsides

- Each chunk has some cutoff text
- Each chunk lacks context

ANTHROPIC

Щоб вирішити ці проблеми, можна додати перекриття між фрагментами. Це означає, що кожен фрагмент включає деякі символи з сусідніх фрагментів, забезпечуючи кращий контекст та гарантуючи цілісність слів і речень.

Workaround: Each chunk gets some overlap with neighboring chunks

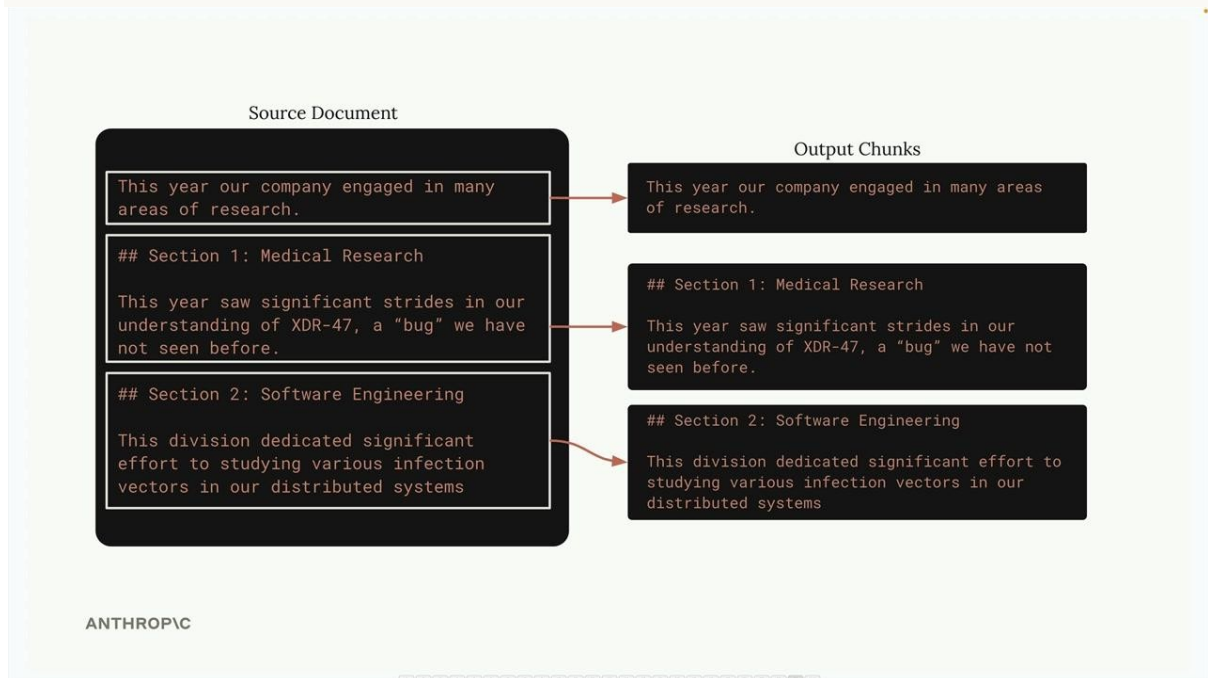


Ось приклад базової реалізації:

```
def chunk_by_char(text, chunk_size=150, chunk_overlap=20):  
    chunks = []  
    start_idx = 0  
  
    while start_idx < len(text):  
        end_idx = min(start_idx + chunk_size, len(text))  
        chunk_text = text[start_idx:end_idx]  
        chunks.append(chunk_text)  
  
        start_idx = (  
            end_idx - chunk_overlap, якщо end_idx < len(text),  
            інакше len(text)  
        )  
  
    return chunks
```

Розбиття на фрагменти на основі структури

Розбиття на фрагменти на основі структури розділяє текст відповідно до природної структури документа — заголовків, абзаців та розділів. Це чудово працює, коли у вас є добре відформатовані документи, такі як файли Markdown.



Для документа у форматі Markdown можна розділяти текст за маркерами заголовків:

```
def chunk_by_section(document_text):
    pattern = r"\n## "

    return re.split(pattern, document_text)
```

Цей підхід дає найчіткіші та найзначущіші фрагменти, оскільки кожен з них представляє повну секцію. Однак він працює лише тоді, коли ви маєте гарантії щодо структури вашого документа. Багато реальних документів — це звичайний текст або PDF-файли без чітких структурних маркерів.

Семантичне розбиття на фрагменти

Семантичне розбиття на фрагменти — це найдосконаліший підхід. Ви поділяєте текст на речення, а потім використовуєте обробку природної мови, щоб визначити, наскільки пов'язані між собою послідовні речення. Ви формуєте фрагменти з груп пов'язаних речень.

Цей метод є обчислювально ресурсоємним, але дає найбільш релевантні фрагменти. Він вимагає розуміння змісту окремих речень і є складнішим у реалізації, ніж інші стратегії.

Розбиття на фрагменти на основі речень

Практичним компромісним варіантом є розбиття на фрагменти за реченнями. Ви розділяєте текст на окремі речення за допомогою регулярних виразів, а потім групуєте їх у фрагменти з можливим перекриттям:

```
def chunk_by_sentence(text, max_sentences_per_chunk=5,
overlap_sentences=1):
    sentences = re.split(r"(?<=[.!?])\s+", text)

    chunks = []
    start_idx = 0

    while start_idx < len(sentences):
        end_idx = min(start_idx + max_sentences_per_chunk,
len(sentences))
        поточний_фрагмент = речення[start_idx:end_idx]
        chunks.append(" ".join(current_chunk))

        start_idx += max_sentences_per_chunk -
overlap_sentences

    якщо start_idx < 0:
        start_idx = 0
```

Вибір стратегії

Ваш вибір повністю залежить від конкретного випадку використання та гарантій щодо документів:

- На основі структури: найкращі результати, коли ви контролюєте форматування документа (наприклад, внутрішні звіти компанії)
- На основі речень: хороший компроміс для більшості текстових документів
- На основі розміру: найнадійніший варіант, що працює з будь-яким типом вмісту, включаючи код

Розбиття на фрагменти за розміром із перекриттям часто є найкращим вибором у виробничому середовищі, оскільки це простий, надійний метод, який працює з будь-яким типом документів. Хоча він може й не давати ідеальних результатів, він стабільно створює прийнятні фрагменти, які не порушують роботу вашого конвеєра.

Пам'ятайте: не існує єдиної «найкращої» стратегії розбиття на фрагменти. Правильний підхід залежить від ваших конкретних документів, сценаріїв використання та компромісів, на які ви готові піти між складністю реалізації та якістю фрагментів.

Вбудовування тексту

Завантаження

002_embeddings.ipynb

(відкривається в новій
вкладці)

Інструкції щодо ключа API VoyageAI.pdf

Після розбиття документа на фрагменти наступним кроком у конвеєрі RAG є визначення, які фрагменти є найбільш релевантними для запиту користувача. По суті, це задача пошуку — потрібно переглянути всі фрагменти тексту та визначити ті, що стосуються запиту користувача.

Finding Relevant Chunks

- This is a search problem!
- How do we find chunks most related to the user's question?

ANTHROPIC

User's Question	How many bugs did engineers fix this year?
------------------------	---

Chunks of Text

This year our company engaged in many areas of research.

Section 1: Medical Research

This year saw significant strides in our understanding of XDR-47, a "bug" we have not seen before.

Section 2: Software Engineering

This division dedicated significant effort to studying various infection vectors in our distributed systems

Семантичний пошук

Найпоширенішим підходом до пошуку релевантних фрагментів є семантичний пошук. На відміну від пошуку за ключовими словами, який шукає точні збіги слів, семантичний пошук використовує вбудовування тексту для розуміння значення та контексту як запиту користувача, так і кожного фрагмента тексту.

Semantic Search

Utilize text embeddings to better understand the user's question *and* what each chunk of text is really talking about.

ANTHROPIC

User's Question	How many bugs did engineers fix this year?
-----------------	--

Chunks of Text

This year our company engaged in many areas of research.

Section 1: Medical Research

This year saw significant strides in our understanding of XDR-47, a "bug" we have not seen before.

Section 2: Software Engineering

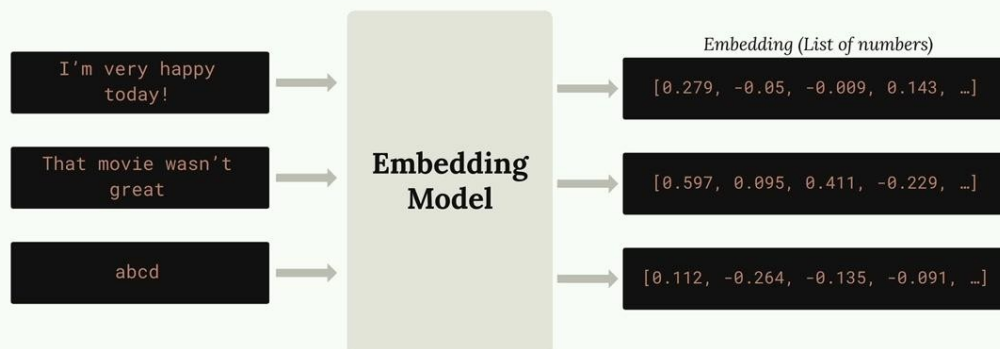
This division dedicated significant effort to studying various infection vectors in our distributed systems

Текстові вбудовування

Текстове вбудовування — це числове представлення змісту, що міститься в тексті. Уявіть собі, що це перетворення слів і речень у формат, з яким комп'ютери можуть працювати математично.

Text Embeddings

A numerical representation of the meaning contained in some text.



ANTHROPIC

Ось як працює цей процес:

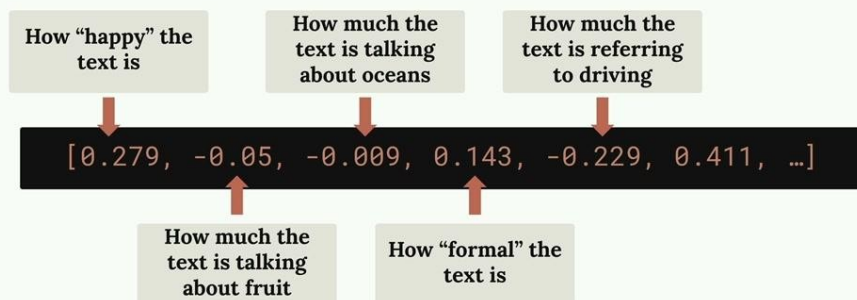
- Ви вводите текст у модель вбудовування
- Модель видає довгий список чисел (вбудовування)
- Кожне число знаходиться в діапазоні від -1 до +1
- Ці числа відображають різні якості або особливості вхідного тексту

Розуміння чисел

Кожне число в вбудовуванні — це, по суті, «оцінка» певної якості вхідного тексту. Однак є важливе застереження: ми не знаємо точно, що саме означає кожне число.

Text Embeddings

Each number is a “score” of some quality of the input text.
We don’t know precisely what each number represents.



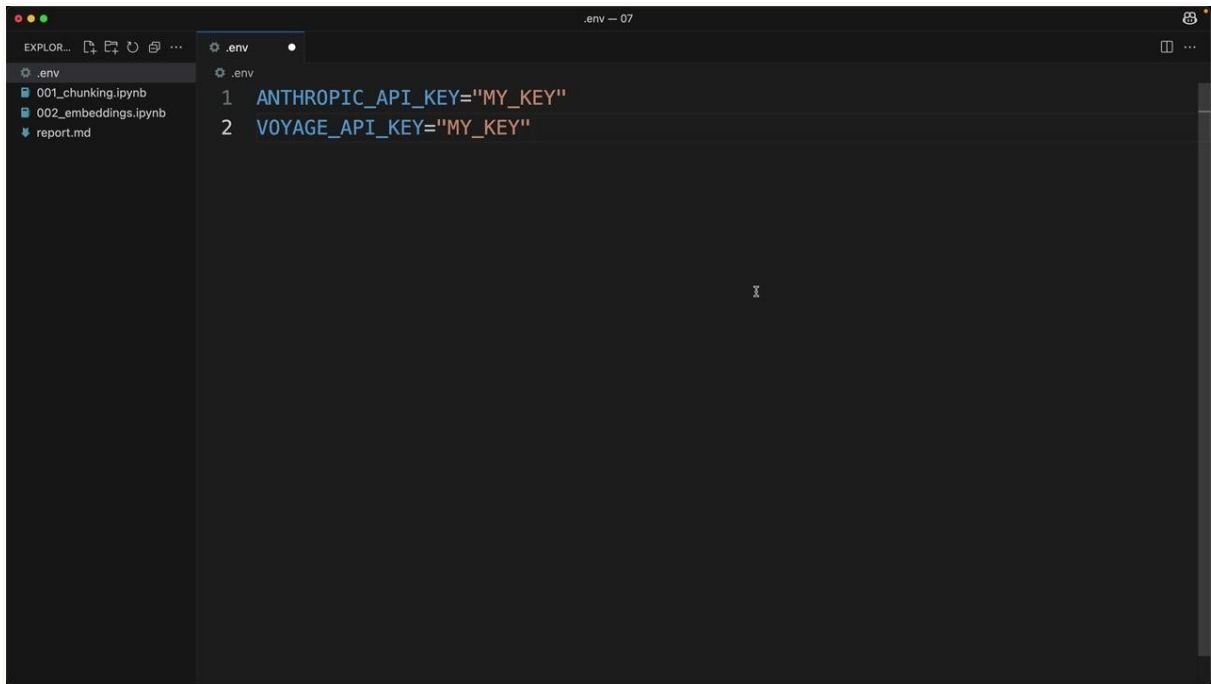
ANTHROPIC

Хоча корисно уявити, що одне число може означати «наскільки текст є радісним» або «наскільки багато в тексті йдеться про океани», це лише концептуальні приклади. Фактичне значення кожного виміру модель засвоює під час навчання, і воно не піддається безпосередньому тлумаченню людиною.

VoyageAI для вбудовувань

Оскільки Anthropic наразі не надає функції генерації вбудовувань, рекомендованим постачальником є VoyageAI. Вам потрібно буде:

- Зареєструвати окремий обліковий запис у VoyageAI
- Отримати ключ API (для початку — безкоштовно)
- Додати ключ до змінних середовища



У файлі **.env** додайте:

```
VOYAGE_API_KEY="ваш_ключ_тут"
```

Реалізація

Спочатку встановіть бібліотеку VoyageAI:

```
%pip install voyageai
```

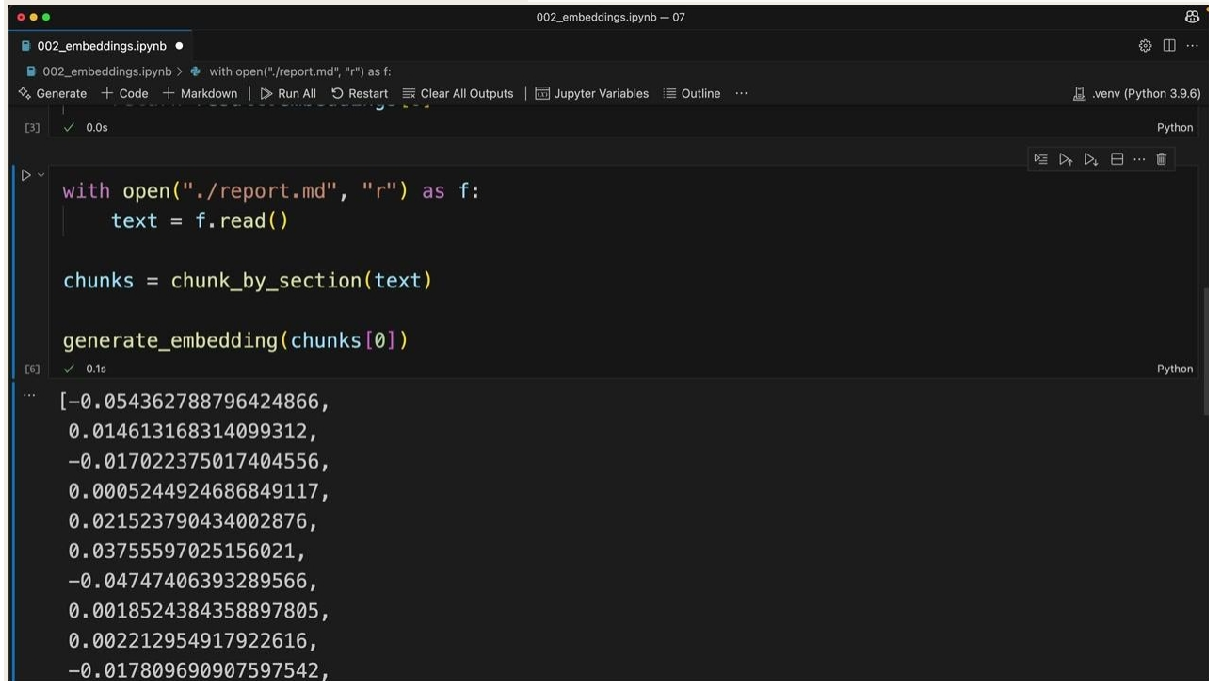
Потім налаштуйте клієнт і створіть функцію для генерації вбудовувань:

```
from dotenv import load_dotenv  
import voyageai
```

```
load_dotenv()  
client = voyageai.Client()
```

```
def generate_embedding(text, model="voyage-3-large",
input_type="query"):
    result = client.embed([text], model=model,
input_type=input_type)

    return result.embeddings[0]
```



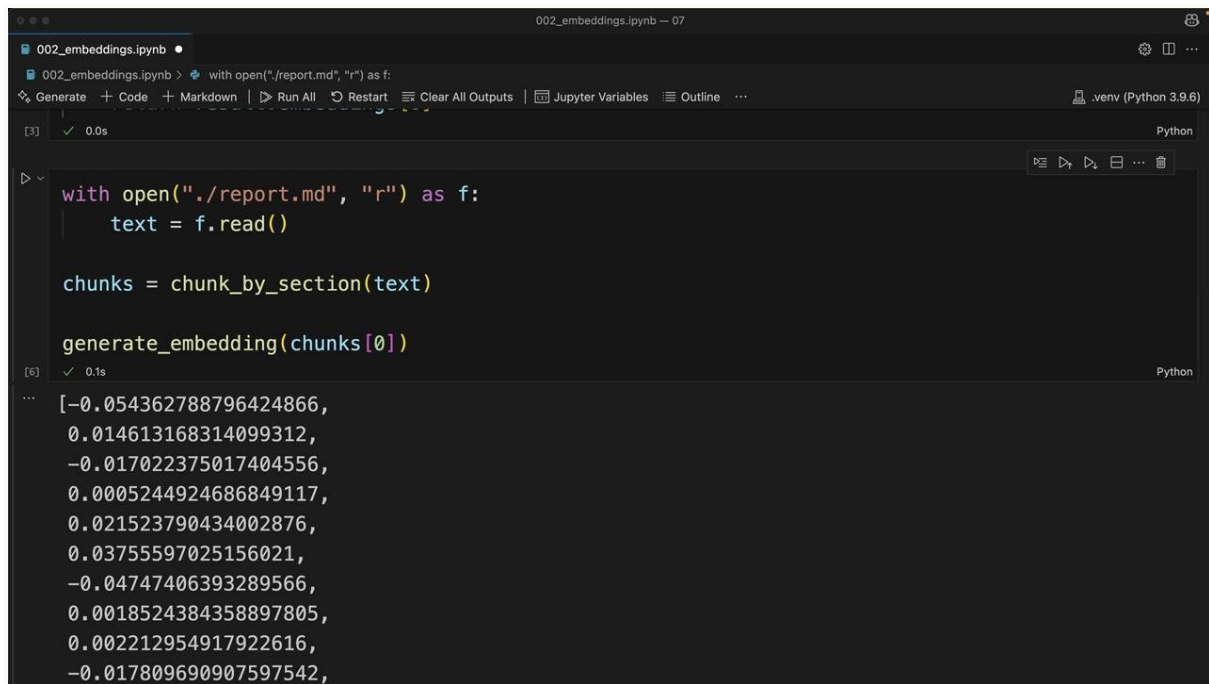
```
002_embeddings.ipynb — 07
002_embeddings.ipynb > with open("./report.md", "r") as f:
    text = f.read()

chunks = chunk_by_section(text)

generate_embedding(chunks[0])

[-0.054362788796424866,
 0.014613168314099312,
 -0.017022375017404556,
 0.0005244924686849117,
 0.021523790434002876,
 0.03755597025156021,
 -0.04747406393289566,
 0.0018524384358897805,
 0.002212954917922616,
 -0.017809690907597542,
```

Коли ви запускаєте цю функцію на фрагменті тексту, ви отримаєте список чисел з плаваючою комою, що представляють вбудовування. Процес є швидким і простим — справжньою проблемою є розуміння того, як ефективно використовувати ці вбудовування у вашому конвеєрі RAG для пошуку найбільш релевантного контенту.



```
002_embeddings.ipynb
002_embeddings.ipynb > with open("./report.md", "r") as f:
    text = f.read()

chunks = chunk_by_section(text)

generate_embedding(chunks[0])
```

```
[5] ✓ 0.1s Python
```

```
... [-0.054362788796424866,
    0.014613168314099312,
    -0.017022375017404556,
    0.0005244924686849117,
    0.021523790434002876,
    0.03755597025156021,
    -0.04747406393289566,
    0.0018524384358897805,
    0.002212954917922616,
    -0.017809690907597542,
```

Наступним кроком є вивчення того, як порівнювати вбудовування, щоб визначити, які фрагменти найбільш схожі на запит користувача, що і становить основу процесу семантичного пошуку.

Завантаження

002_embeddings.ipynb
(відкривається в новій
вкладці)

Інструкції щодо ключа API
VoyageAI.pdf (відкривається в новій
вкладці)

Повний потік RAG

Тепер, коли ми розглянули основи RAG, поділу тексту на фрагменти та вбудовування, давайте крок за кроком пройдемося по всьому конвеєру RAG. Цей приклад покаже вам, як саме всі ці елементи взаємодіють між собою для пошуку релевантної інформації та формування відповідей.

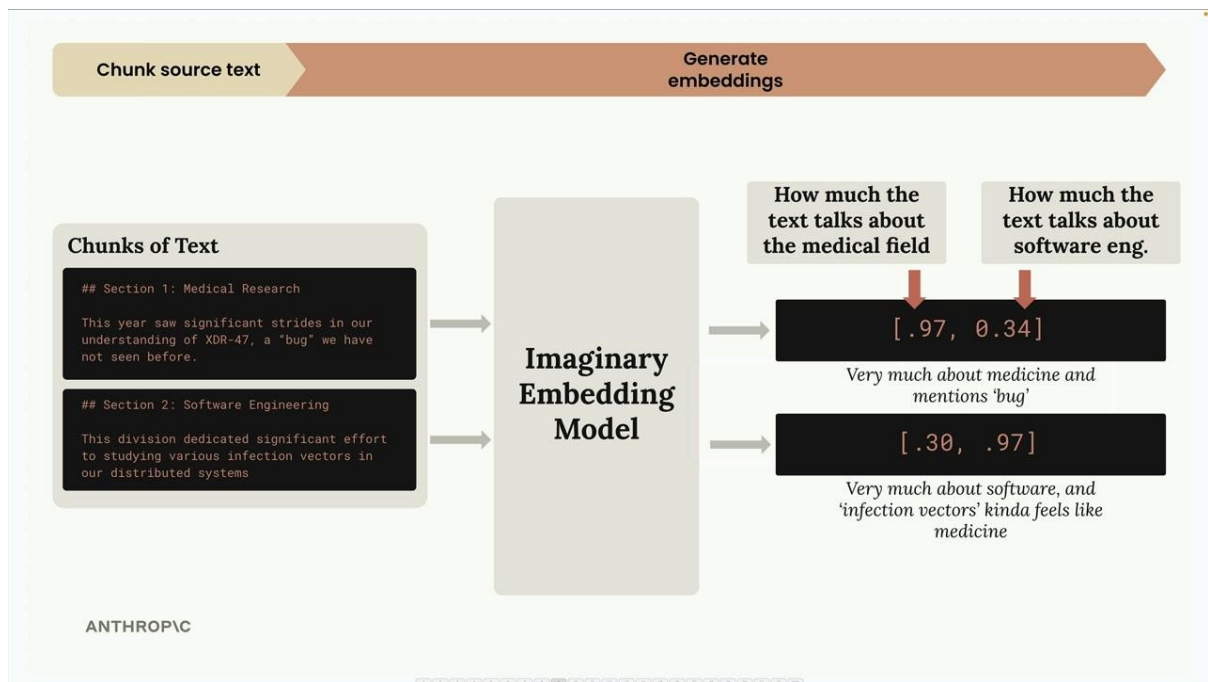
Крок 1: Розбиття вихідного тексту на фрагменти

Спочатку ми беремо вихідний документ і розбиваємо його на зручні для обробки фрагменти. У цьому прикладі ми використаємо два прості уривки тексту:

- Розділ 1: Медичні дослідження — «Цього року відбулися значні прориви в нашому розумінні XDR-47, «вірусу», якого ми раніше не бачили».
- Розділ 2: Розробка програмного забезпечення — «Цей підрозділ доклав значних зусиль до вивчення різних векторів зараження в наших розподілених системах»

Крок 2: Генерування вбудовувань

Далі ми перетворюємо кожен фрагмент тексту на числові вбудовування за допомогою моделі вбудовування. Щоб це було легше зрозуміти, уявімо, що ми маємо ідеальну модель вбудовування, яка завжди повертає саме два числа, і ми знаємо, що означає кожне з них.



У нашій уявній моделі:

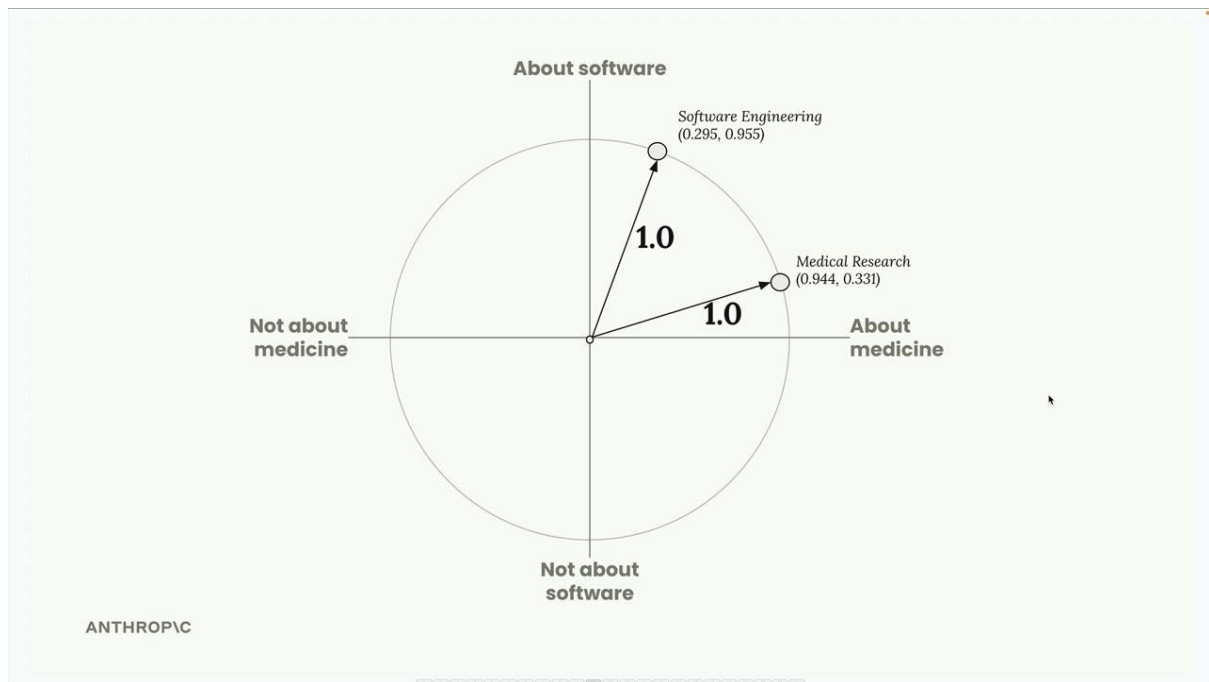
- Перше число показує, наскільки текст стосується медичної галузі
- Друге число показує, наскільки текст стосується програмної інженерії

Для розділу про медичні дослідження ми можемо отримати $[0,97; 0,34]$ — текст дуже зосереджений на медицині, але містить деякі елементи, пов'язані з програмним забезпеченням, завдяки слову «bug». Для розділу про розробку програмного забезпечення ми отримуємо $[0,30; 0,97]$ — текст значною мірою зосереджений на програмному забезпеченні, але містить медичні відтінки завдяки виразу «вектори інфекції».

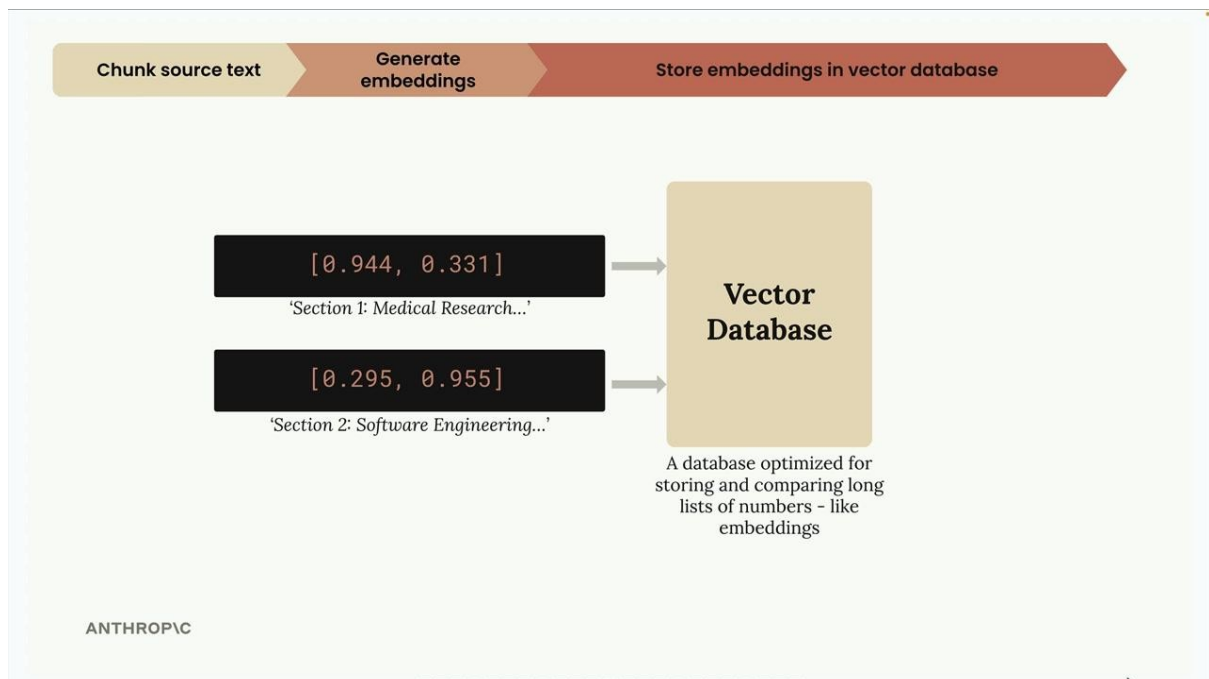
Нормалізація

API вбудовування зазвичай виконує етап нормалізації, який масштабує кожен вектор так, щоб його величина дорівнювала 1,0. Вам не потрібно турбуватися про

математику — це відбувається автоматично. У результаті ми отримуємо нормалізовані вектори, такі як $[0,944; 0,331]$ та $[0,295; 0,955]$.

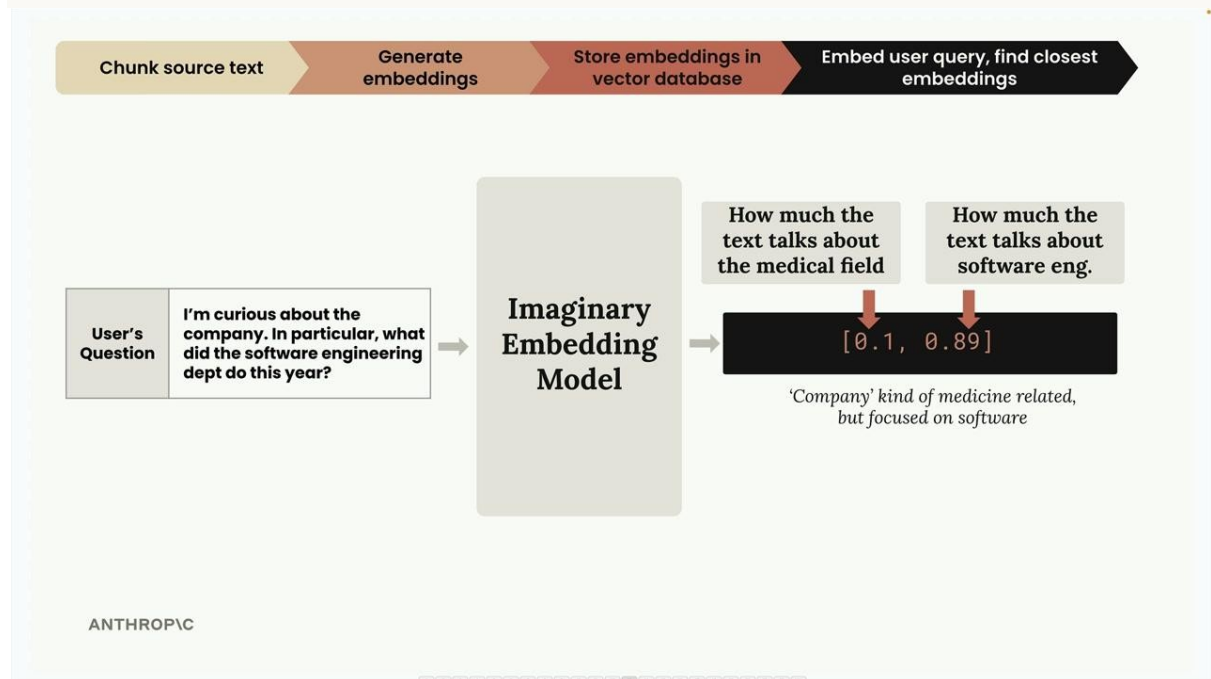


Ми можемо візуалізувати ці вбудовування на одиничному колі, де кожна точка представляє один із наших фрагментів тексту.



Крок 3: Збереження у векторній базі даних

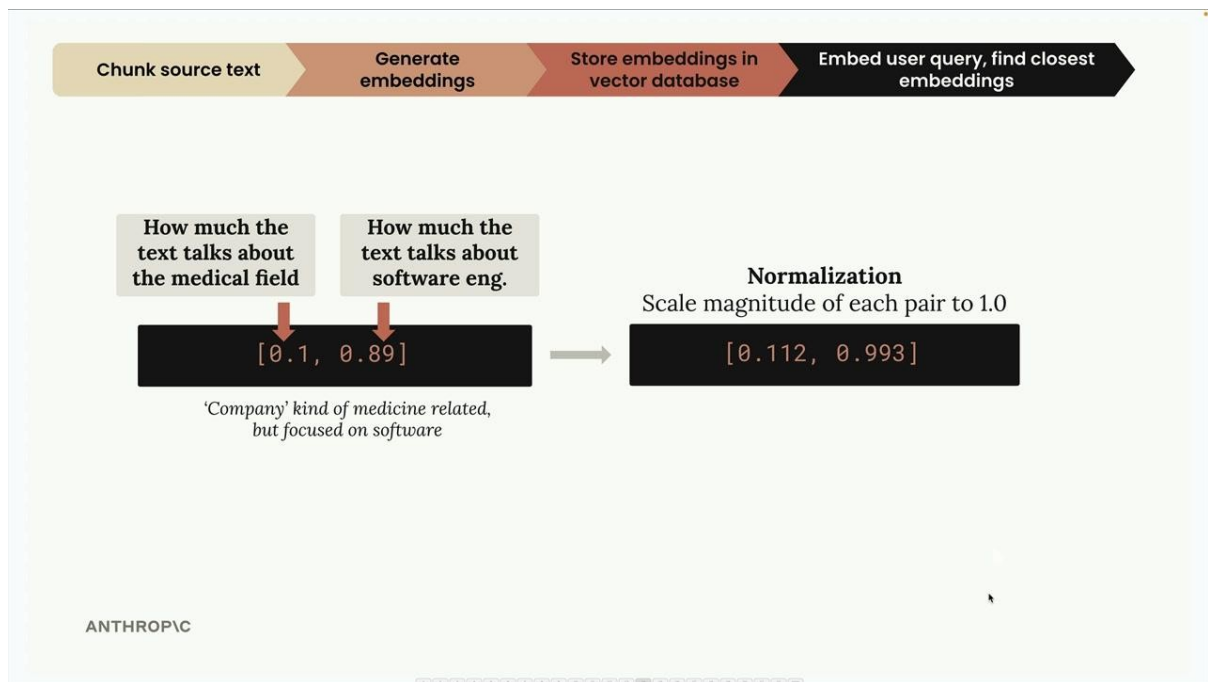
Ми зберігаємо ці вбудовування у векторній базі даних — спеціалізованій базі даних, оптимізованій для зберігання, порівняння та пошуку у довгих списках чисел, таких як наші вбудовування.



На цьому етапі ми робимо паузу. Уся робота, виконана до цього моменту, — це попередня обробка, яка відбувається заздалегідь. Тепер ми чекаємо, поки користувач надішле запит.

Крок 4: Обробка запиту користувача

Коли користувач задає питання на кшталт «Мені цікава ця компанія. Зокрема, чим займався відділ розробки програмного забезпечення цього року?», ми обробляємо його запит за допомогою тієї самої моделі вбудовування.

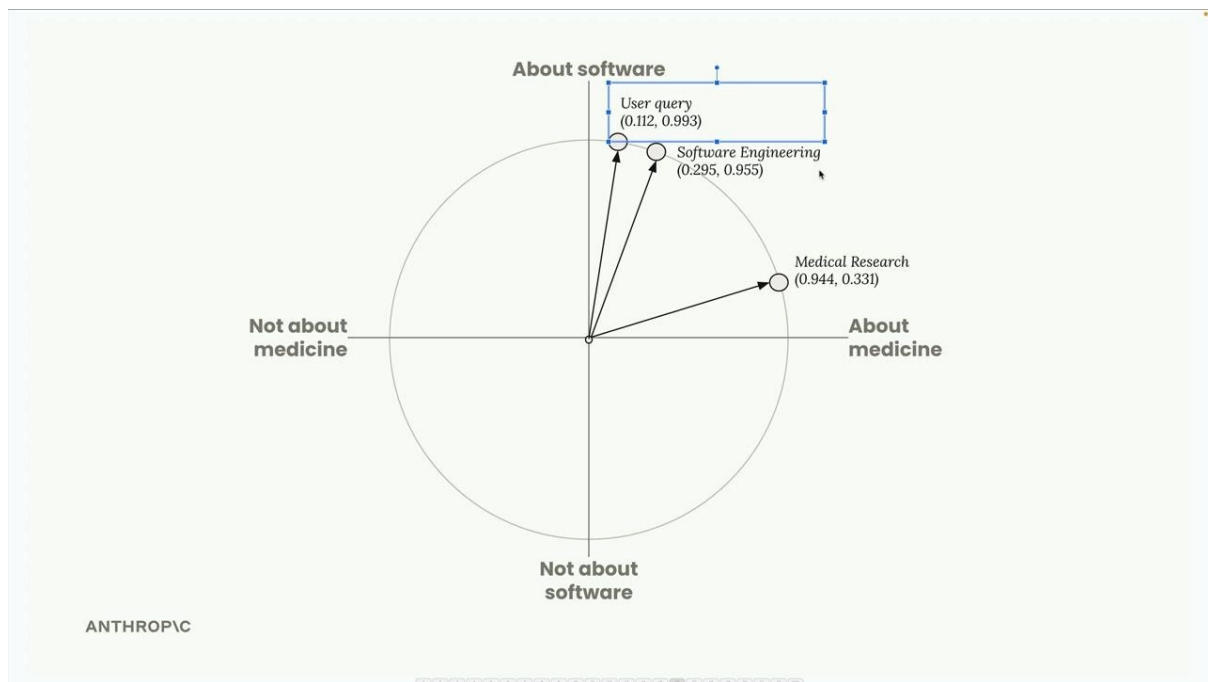


Цей запит вбудовується як щось на зразок $[0,1; 0,89]$ — низький оцінка за медичною тематикою, висока оцінка за програмною інженерією. Після нормалізації ми отримуємо $[0,112; 0,993]$.

Крок 5: Пошук подібних вбудовувань

Ми надсилаємо вбудовування запиту користувача до нашої векторної бази даних і просимо її

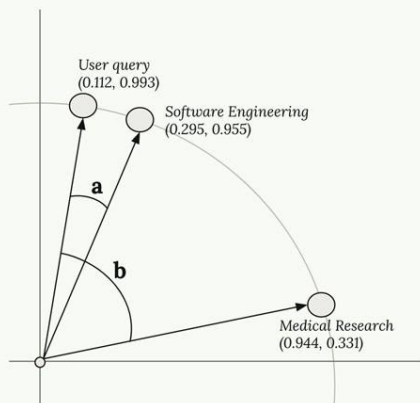
знайти найбільш схожі збережені вбудовування.



База даних повертає розділ «програмна інженерія», оскільки він є найближче відповідає тому, про що запитував користувач.

Як працює визначення схожості: косинусна схожість

Векторна база даних використовує косинусну схожість, щоб визначити, які вбудовування є найбільш схожими. Цей показник вимірює косинус кута між двома векторами.



Cosine Similarity

- Cosine of the angle between the two vectors
- If they're in the exact same direction we'd get 1.0
- If perpendicular, 0.0
- If completely opposite, -1.0

$$\cos(a) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} = \frac{(0.112, 0.993) \cdot (0.295, 0.955)}{\|(0.112, 0.993)\| \cdot \|(0.295, 0.955)\|} = 0.983$$

$$\cos(b) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} = \frac{(0.112, 0.993) \cdot (0.944, 0.331)}{\|(0.112, 0.993)\| \cdot \|(0.944, 0.331)\|} = 0.398$$

Основні моменти щодо косинусної схожості:

- Результати коливаються в діапазоні від -1 до 1
- Значення, близькі до 1, означають високу схожість
- Значення, близькі до -1, означають значну відмінність
- 0 означає перпендикулярність (відсутність зв'язку)

У нашому прикладі косинусна схожість між запитом користувача та фрагментом про програмну інженерію становить 0,983 — дуже висока схожість. Схожість із фрагментом про медичні дослідження становить лише 0,398 — значно нижча.

Косинусна відстань

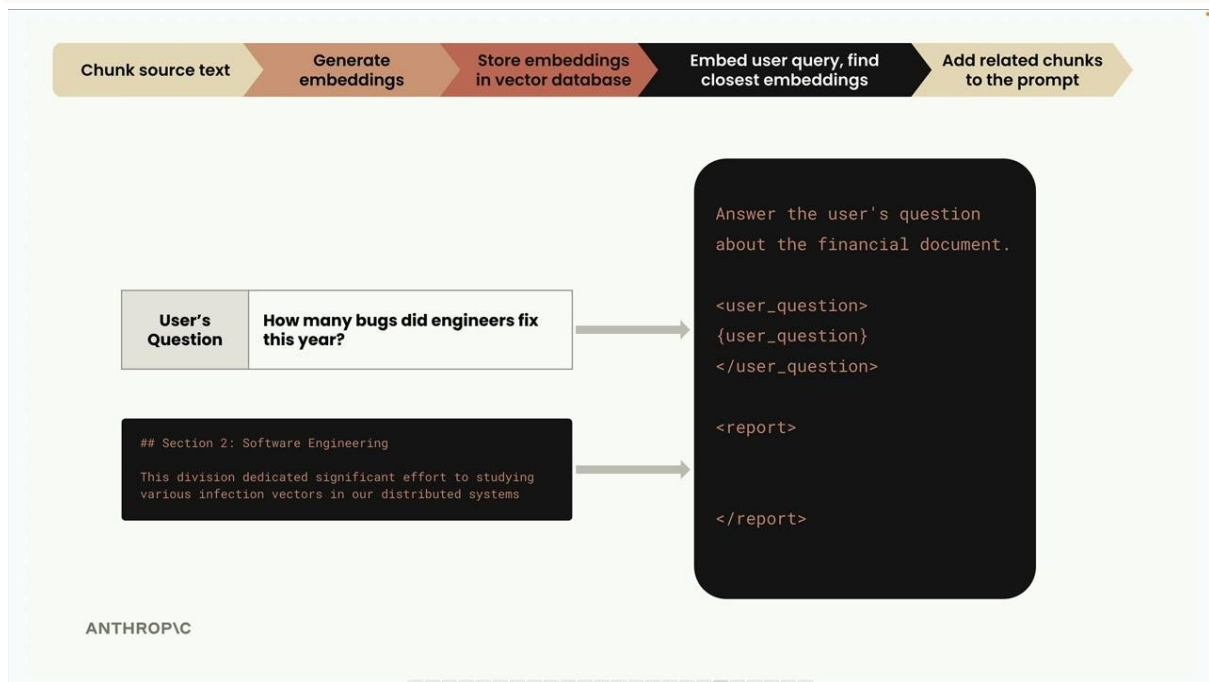
У документації до векторних баз даних часто зустрічається поняття «косинусна відстань». Вона обчислюється просто як **(1 — косинусна схожість)**. З косинусною відстанню:

- Значення, близькі до 0, означають високу схожість
- Більші значення означають меншу схожість

Таке коригування полегшує інтерпретацію чисел у багатьох контекстах.

Крок 6: Створення остаточного запиту

Нарешті, ми беремо запитання користувача та найрелевантніший фрагмент тексту, який ми знайшли, об'єднуємо їх у запит і надсилаємо його до Claude для отримання відповіді.



Запит може виглядати так:

Дайте відповідь на запитання користувача щодо фінансового документа.

```
<user_question>
```

Скільки помилок виправили інженери цього року?

```
</user_question>
```

```
<report>
```

```
## Розділ 2: Розробка програмного забезпечення
```

Цей підрозділ доклав значних зусиль до вивчення різних векторів зараження в наших розподілених системах

</report>

І ось повний конвеєр RAG! Система успішно витягла найбільш релевантну інформацію на основі семантичної схожості та надала її як контекст для формування точної відповіді.

Впровадження конвеєра RAG

Завантаження

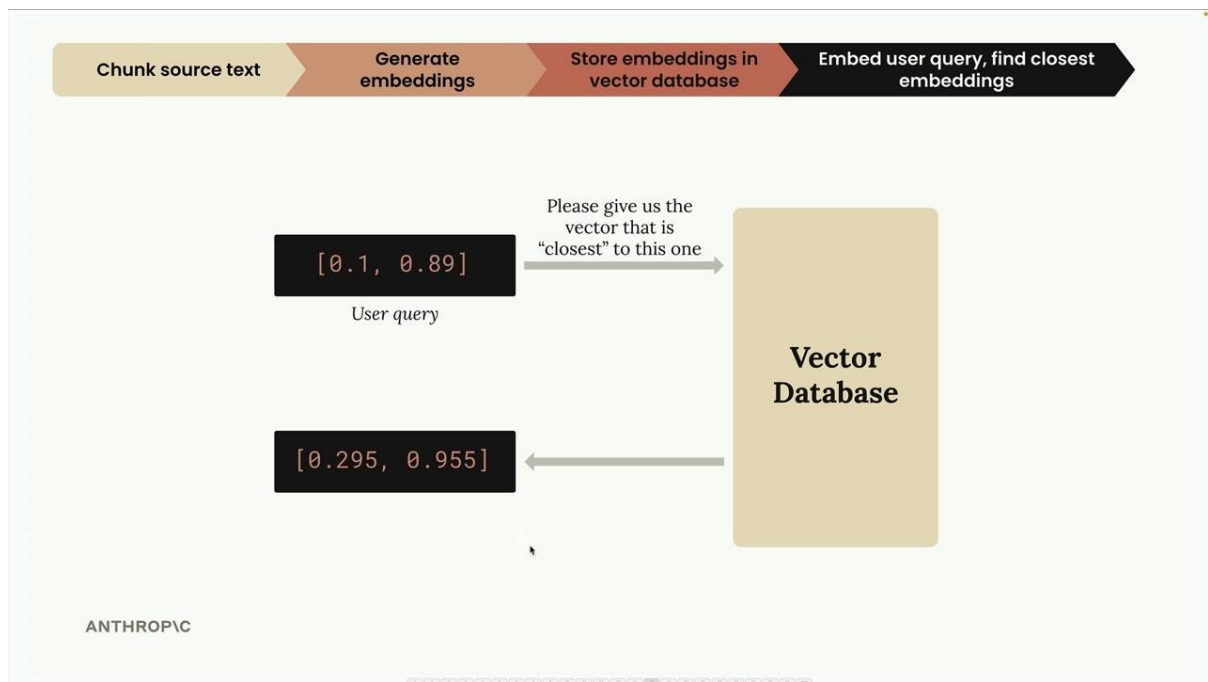
003_vectordb.ipynb

Тепер, коли ми концептуально зрозуміли алгоритм RAG, давайте реалізуємо його крок за кроком. Ми розглянемо повний приклад, який демонструє, як розбивати текст на фрагменти, генерувати вбудовування, зберігати їх у векторній базі даних та виконувати пошук за схожістю.

Впровадження методу RAG у п'ять етапів

Наша реалізація складається з тих самих п'яти кроків, які ми розглядали раніше:

1. Розбиття тексту на фрагменти за розділами
2. Генеруємо вбудовування для кожного фрагмента
3. Створення векторного сховища та додавання до нього кожного вбудовування
4. Генеруємо вбудовування для запиту користувача
5. Здійснити пошук у сховищі, щоб знайти найбільш релевантні фрагменти



Ця діаграма показує, як ми перетворюємо запити користувачів на вбудовування та здійснюємо пошук у нашій векторній базі даних, щоб знайти найбільш релевантний контент.

Крок 1: Розбиття тексту на фрагменти

Спочатку ми завантажуюмо документ і розбиваємо його на зручні для обробки фрагменти:

```
with open("./report.md", "r") as f:
    text = f.read()
```

```
chunks = chunk_by_section(text)
```

```
chunks[2]    # Перевіряємо, чи з'явився зміст
```

Ми використовуємо ту саму функцію **chunk_by_section**, що й раніше, щоб розділити наш документ на логічні розділи.

Крок 2: Генеруємо вбудовування

Далі ми створюємо вбудовування для всіх наших фрагментів одночасно:

```
embeddings = generate_embedding(chunks)
```

Функція вбудовування була оновлена, щоб обробляти як окремі рядки, так і списки рядків, що робить її більш ефективною для пакетної обробки.

Крок 3: Збереження у векторній базі даних

Тепер ми створюємо наш векторний масив і заповнюємо його вбудовуваннями та пов'язаним з ними текстом:

```
store = VectorIndex()
```

```
for embedding, chunk in zip(embeddings, chunks):
```

```
    store.add_vector(embedding, {"content": chunk})
```

Зверніть увагу, що ми зберігаємо як вбудовування, так і оригінальний текстовий вміст. Це дуже важливо, оскільки під час подальшого пошуку нам потрібно повертати саме текст, а не лише числові значення вбудовувань.

Навіщо зберігати оригінальний текст?

Під час запиту до нашої векторної бази даних отримання лише чисел вбудовування не є корисним. Нам потрібен власне текст, який використовувався для генерації цих вбудовувань. Ось чому ми включаємо оригінальний текст фрагмента (або принаймні посилання на нього) разом із кожним вбудовуванням у нашої бази даних.

Крок 4: Обробка запитів користувачів

Коли користувач задає питання, ми генеруємо вбудовування для його запиту:

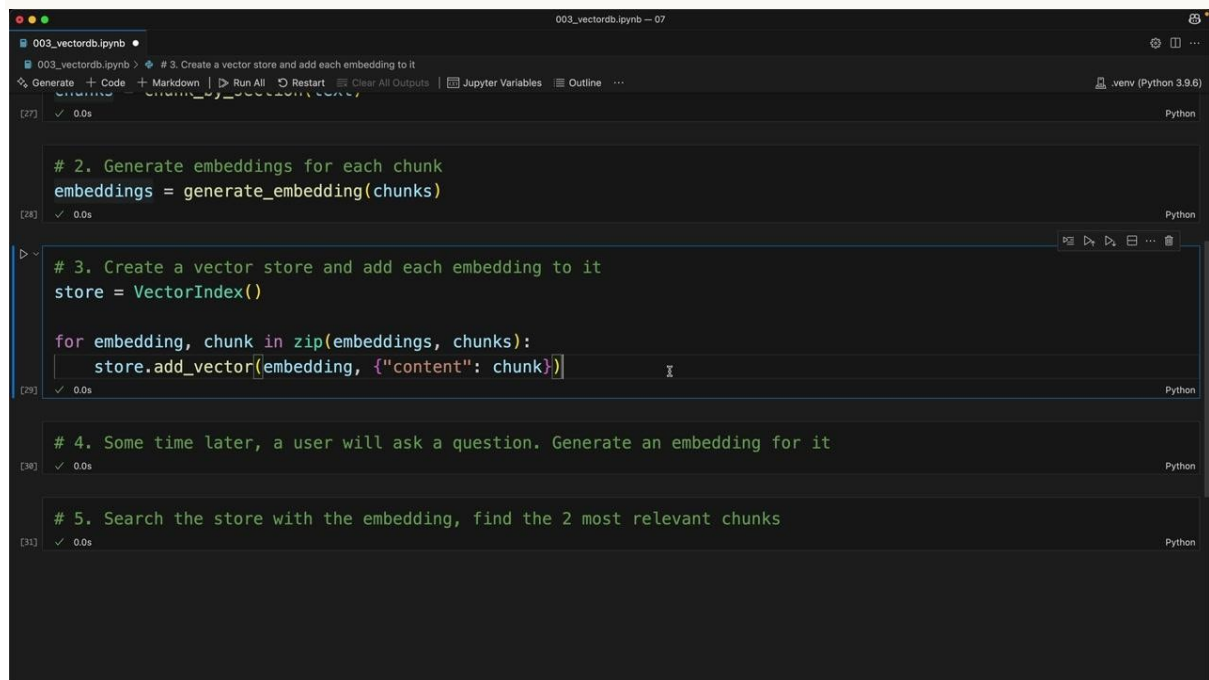
```
user_embedding = generate_embedding("Чим займався відділ програмної інженерії  
минулого року?")
```

Крок 5: Пошук релевантного контенту

Нарешті, ми здійснюємо пошук у нашому векторному сховищі, щоб знайти найбільш схожі фрагменти:

```
results = store.search(user_embedding, 2) for  
  
doc, distance in results:  
  
    print(distance, "\n", doc["content"][0:200], "\n")
```

Цей пошук повертає два найрелевантніші фрагменти разом із їхніми показниками схожості (косинусними відстанями).



The screenshot shows a Jupyter Notebook titled '003_vectordb.ipynb' with the following code cells:

```
# 2. Generate embeddings for each chunk
embeddings = generate_embedding(chunks)

# 3. Create a vector store and add each embedding to it
store = VectorIndex()

for embedding, chunk in zip(embeddings, chunks):
    store.add_vector(embedding, {"content": chunk})

# 4. Some time later, a user will ask a question. Generate an embedding for it

# 5. Search the store with the embedding, find the 2 most relevant chunks
```

Результати пошуку показують, які розділи нашого документа є найбільш релевантними для запиту користувача, разом із показниками схожості.

Розуміння результатів

Коли ми запускаємо наш приклад запиту про відділ програмної інженерії, ми отримуємо:

- Розділ 2: Програмна інженерія з відстанню 0,71 (найближчий збіг)
- Розділ «Методологія» з відстанню 0,72 (другий за близькістю)

Чим нижче значення відстані, тим вища схожість, тому розділ 2 є найбільш релевантним для нашого запиту.

Що далі?

Ця реалізація добре працює в базових випадках, але є сценарії, в яких вона не працює так, як очікується. У наступних розділах ми розглянемо вдосконалення, які зроблять нашу систему RAG більш надійною та точною.

Головний висновок полягає в тому, що суть RAG полягає в перетворенні тексту на числа (вбудовування), ефективному зберіганні цих чисел, а потім використанні математичної схожості для пошуку релевантного контенту, коли користувачі задають запитання.

Лексичний пошук BM25

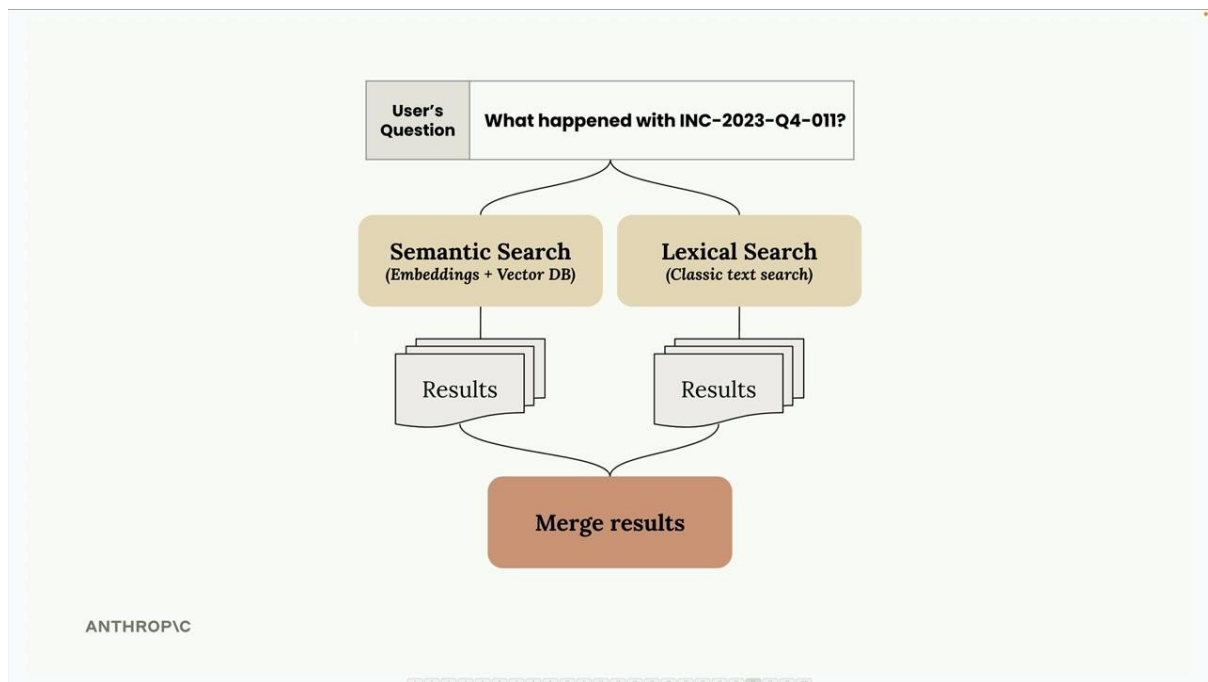
Завантаження

004_bm25.ipynb

Під час побудови RAG-конвеєрів ви швидко зрозумієте, що семантичний пошук сам по собі не завжди дає найкращі результати. Іноді потрібні точні збіги термінів, які семантичний пошук може пропустити. Рішенням є поєднання семантичного пошуку з лексичним за допомогою методу, відомого як BM25.

Проблема використання лише семантичного пошуку

Припустимо, ви шукаєте конкретний ідентифікатор інциденту, наприклад «INC-2023-Q4-011» у документі. Хоча семантичний пошук чудово розуміє контекст і значення, він може повернути розділи, які семантично пов'язані, але насправді не містять точного терміна, який ви шукаєте.

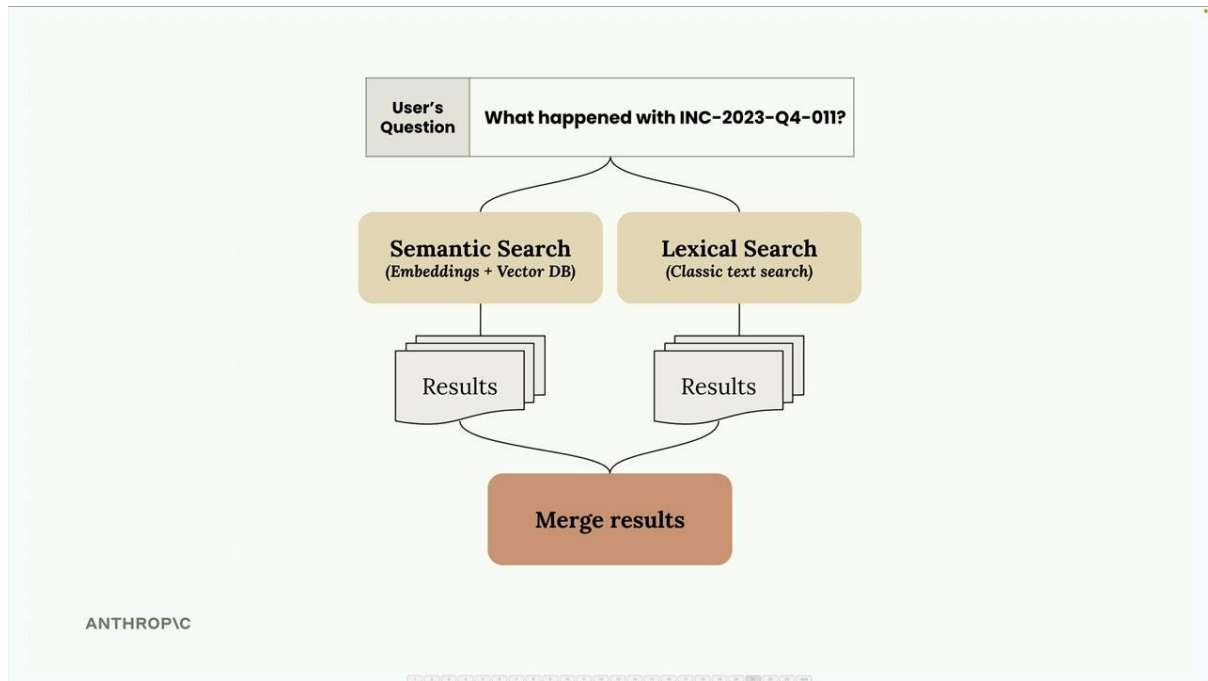


У наведеному вище прикладі семантичний пошук повернув розділ про кібербезпеку (який дійсно містить ідентифікатор інциденту), але також повернув розділ фінансового аналізу, в якому інцидент взагалі не згадується. Це відбувається тому, що семантичний пошук зосереджується на концептуальній схожості, а не на точній відповідності термінів.

Стратегія гібридного пошуку

Вирішенням є паралельне виконання семантичного та лексичного пошуків, а потім

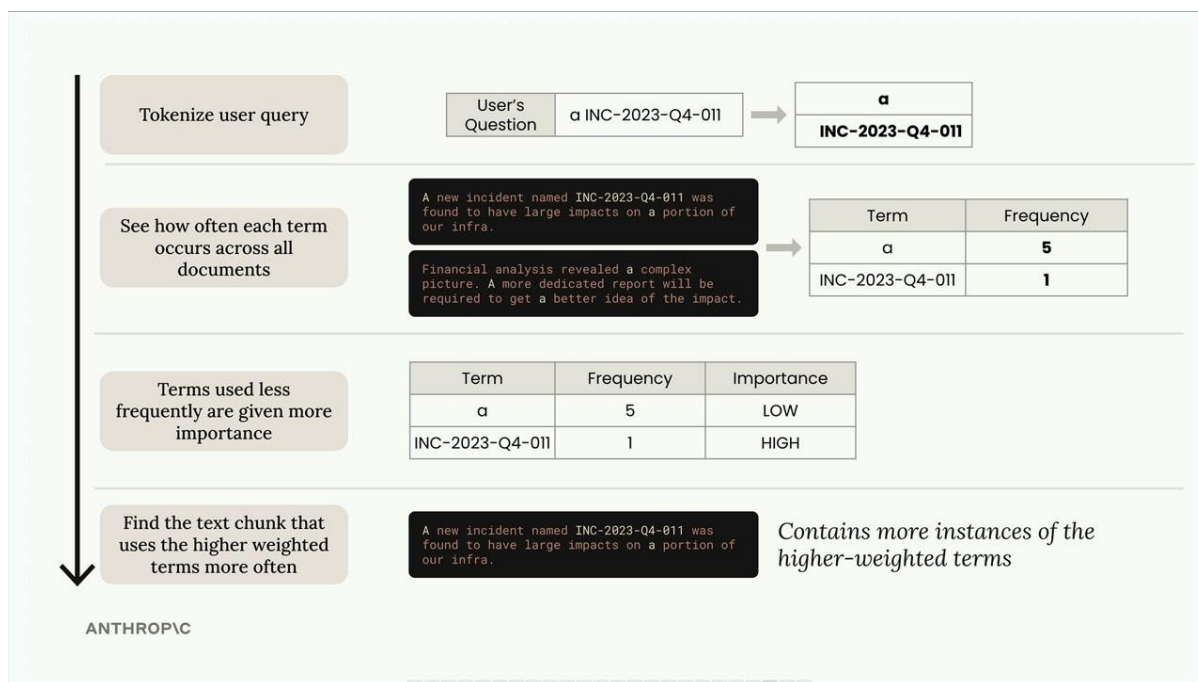
об'єднати результати. Це дозволяє отримати найкраще з обох підходів:



- Семантичний пошук знаходить концептуально пов'язаний контент за допомогою вбудованих векторів
- Лексичний пошук знаходить точні збіги термінів за допомогою класичного текстового пошуку
- Об'єднані результати поєднують обидва підходи для більшої точності

Як працює BM25

BM25 (Best Match 25) — це популярний алгоритм лексичного пошуку в системах RAG. Ось як він обробляє пошуковий запит:



Крок 1: Розбиття запиту на токени

Розбийте запит користувача на окремі терміни. Наприклад, «a INC-2023-Q4-011» перетворюється на ["a", "INC-2023-Q4-011"].

Крок 2: Підрахунок частоти термінів

Визначається, як часто кожен термін зустрічається у всіх ваших документах. Загальні слова, такі як «a», можуть зустрічатися 5 разів, тоді як конкретні терміни, такі як «INC-2023-Q4-011», — лише один раз.

Крок 3: Присвоєння ваги термінам за важливістю

Терміни, що зустрічаються рідше, отримують вищі оцінки важливості. Слово «a» отримує низьку оцінку важливості, оскільки воно є загальним, тоді як «INC-2023-Q4-011» отримує високу оцінку важливості, оскільки воно рідкісне.

Крок 4: Пошук найкращих збігів

Повернути документи, що містять більше вживань термінів із вищим коефіцієнтом важливості.

Реалізація пошуку за алгоритмом BM25

Ось як налаштувати базову систему пошуку BM25:

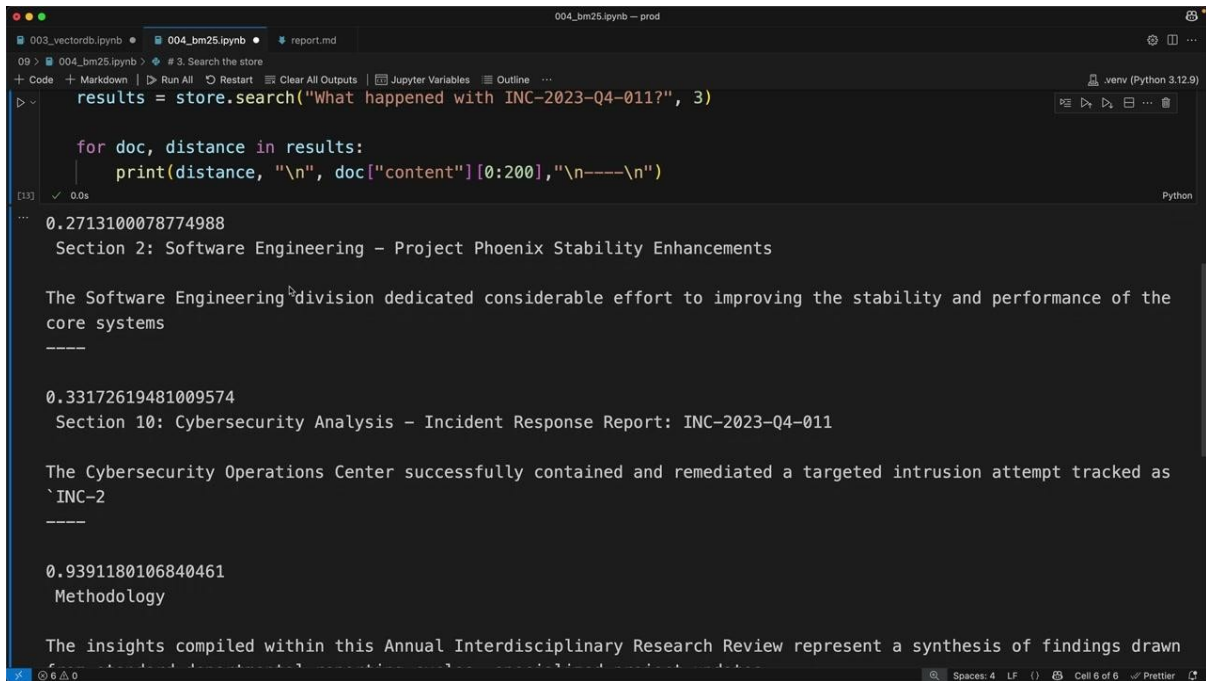
```
# 1. Розбийте текст на розділи chunks =
chunk_by_section(text)

# 2. Створіть індекс BM25 та додайте документи store =
BM25Index()
for chunk in chunks:
    store.add_document({"content": chunk})

# 3. Здійсніть пошук у сховищі
results = store.search("Що сталося з INC-2023-Q4-011?", 3)

# Вивести результати
for doc, distance in results:
    print(distance, "\n", doc["content"][:200], "\n-----\n")
```

Якщо запустити цей пошук, ви отримаєте набагато кращі результати, ніж при використанні лише семантичного пошуку. Алгоритм BM25 надає пріоритет розділам, які насправді містять ваші конкретні пошукові терміни, особливо рідкісні терміни, такі як ідентифікатори інцидентів.



The screenshot shows a Jupyter Notebook interface with a dark theme. The code cell contains a search query and a loop that prints the results. The output shows three results, each with a distance score and a snippet of text. The first result is 'Section 2: Software Engineering - Project Phoenix Stability Enhancements'. The second result is 'Section 10: Cybersecurity Analysis - Incident Response Report: INC-2023-Q4-011'. The third result is 'Methodology'.

```
results = store.search("What happened with INC-2023-Q4-011?", 3)

for doc, distance in results:
    print(distance, "\n", doc["content"][0:200], "\n----\n")
```

0.2713100078774988
Section 2: Software Engineering - Project Phoenix Stability Enhancements

The Software Engineering division dedicated considerable effort to improving the stability and performance of the core systems

0.33172619481009574
Section 10: Cybersecurity Analysis - Incident Response Report: INC-2023-Q4-011

The Cybersecurity Operations Center successfully contained and remediated a targeted intrusion attempt tracked as INC-2

0.9391180106840461
Methodology

The insights compiled within this Annual Interdisciplinary Research Review represent a synthesis of findings drawn

Зверніть увагу, як результати тепер правильно надають пріоритет розділам «Розробка програмного забезпечення» та «Кібербезпека» — обидва вони насправді містять ідентифікатор інциденту, який ви шукаєте.

Чому це працює краще

BM25 чудово справляється з пошуком точних збігів, оскільки:

- Надає більшу вагу рідкісним, специфічним термінам
- Ігнорує загальні слова, що не додають цінності пошуку
- Зосереджується на частоті вживання терміна, а не на його семантичному значенні
- Особливо добре працює з технічними термінами, ідентифікаторами та конкретними фразами

Головний висновок полягає в тому, що обидва методи пошуку мають взаємодоповнюючі переваги. Семантичний пошук розуміє контекст і значення, тоді як лексичний пошук гарантує, що ви не пропустите точні збіги термінів.

Поєднуючи їх, ви

створюєте більш надійну систему пошуку, яка ефективно обробляє як концептуальні запити

, і конкретні запити.

На наступному етапі ви дізнаєтеся, як об'єднати результати з обох пошукових

систем, щоб створити єдиний гібридний пошук.

Конвеєр RAG з кількома індексами

Завантаження

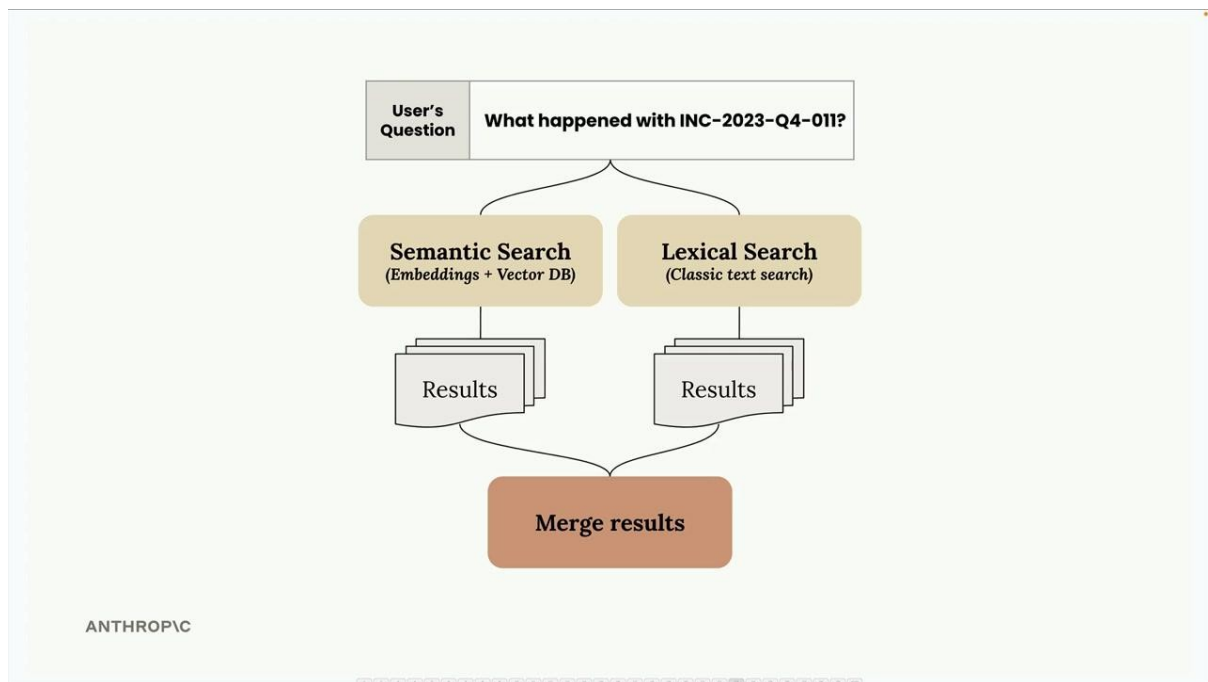
005_hybrid.ipynb

Ми створили окремі реалізації для семантичного пошуку (з використанням векторних вбудовувань) та лексичного пошуку (з використанням BM25). Тепер настав час об'єднати їх у єдиний пошуковий конвеєр, який використовує переваги обох підходів.

Архітектура з кількома індексами

Наші класи `VectorIndex` та `BM25Index` мають майже ідентичні API

— обидва мають методи `add_document()` та `search()`. Ця узгодженість дозволяє легко об'єднати їх у новий клас під назвою `Retriever`.



Retriever виступає координатором, який пересилає запити користувачів до обох індексів, збирає їхні результати та об'єднує їх за допомогою техніки, що називається взаємним злиттям рейтингів.

Що таке взаємне злиття рейтингів

Об'єднання результатів, отриманих різними методами пошуку, — це не просто з'єднання списків. Кожен метод використовує різні системи оцінювання, тому нам потрібен спосіб справедливо нормалізувати та об'єднати їхні рейтинги.

Reciprocal Rank Fusion

Combine outputs based upon the **reciprocal rank** of each text chunk

VectorIndex

Rank	Text Chunk	Score
1	Section 2: Software Engineering...	0.234
2	Section 7: Historical Research...	0.913
3	Section 6: Product Engineering	1.327

BM25Index

Rank	Text Chunk	Score
1	Section 6: Product Engineering	0.184
2	Section 2: Software Engineering...	0.414
3	Section 7: Historical Research...	0.781

ANTHROPIC

Ось як працює взаємне злиття рейтингів на прикладі. Припустимо, ми шукаємо інформацію про «INC-2023-Q4-011» і отримуємо такі результати:

- VectorIndex повертає: Розділ 2 (рейтинг 1), Розділ 7 (рейтинг 2), Розділ 6 (рейтинг 3)
- BM25Index повертає: Розділ 6 (рейтинг 1), Розділ 2 (рейтинг 2), Розділ 7 (рейтинг 3)

VectorIndex

Rank	Text Chunk
1	Section 2: Software Engineering...
2	Section 7: Historical Research...
3	Section 6: Product Engineering

BM25Index

Rank	Text Chunk
1	Section 6: Product Engineering
2	Section 2: Software Engineering...
3	Section 7: Historical Research...

Text Chunk	Rank From Vector Index	Rank from BM25 Index
Section 2: Software Engineering...	1	2
Section 7: Historical Research...	2	3
Section 6: Product Engineering...	3	1

ANTHROPIC

Ми об'єднуємо ці дані в єдину таблицю, що показує рейтинг кожного фрагмента тексту за обома індексами, а потім застосовуємо формулу RRF:

$$\text{RRF_score}(d) = \sum (1 / (k + \text{rank}_i(d)))$$

Де k — константа (зазвичай 60, але для наочності ми використаємо 1), а $\text{rank}_i(d)$ — рейтинг документа d в i -му рейтингу.

$$\text{RRF_score}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}$$

k is a constant, 60 is often used, but going to use 1 here for clearer results

Text Chunk	Rank From Vector	Rank from BM25	Score
Section 2: Software Engineering...	1	2	$1.0 / (1 + 1) + 1.0 / (1 + 2) = \mathbf{0.833}$
Section 7: Historical Research...	2	3	$1.0 / (1 + 2) + 1.0 / (1 + 3) = \mathbf{0.583}$
Section 6: Product Engineering...	3	1	$1.0 / (1 + 3) + 1.0 / (1 + 1) = \mathbf{0.75}$

Higher score = more relevant text chunk

ANTHROPIC

Для нашого прикладу:

- Розділ 2: $1,0/(1+1) + 1,0/(1+2) = 0,833$
- Розділ 7: $1,0/(1+2) + 1,0/(1+3) = 0,583$
- Розділ 6: $1,0/(1+3) + 1,0/(1+1) = 0,75$

$$\text{RRF_score}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}$$

k is a constant, 60 is often used, but going to use 1 here for clearer results

Text Chunk	Rank From Vector	Rank from BM25	Score	Final Rank
Section 2: Software Engineering...	1	2	$1.0 / (1 + 1) + 1.0 / (1 + 2) = \mathbf{0.833}$	1
Section 6: Product Engineering...	3	1	$1.0 / (1 + 3) + 1.0 / (1 + 1) = \mathbf{0.75}$	2
Section 7: Historical Research...	2	3	$1.0 / (1 + 2) + 1.0 / (1 + 3) = \mathbf{0.583}$	3

Higher score = more relevant text chunk

ANTHROPIC

Остаточний рейтинг виглядає так: Розділ 2 (0,833), Розділ 6 (0,75), Розділ 7 (0,583). Це інтуїтивно зрозуміло — Розділ 2 показав хороші результати за обома показниками, тому він посідає перше місце.

Деталі реалізації

Клас `Retriever` об'єднує кілька пошукових індексів і надає єдиний інтерфейс:

```
class Retriever:
    def __init__(self, *indexes: SearchIndex):
        if len(indexes) == 0:
            raise ValueError("Необхідно вказати хоча б один індекс")
        self._indexes = list(indexes)

    def add_document(self, document: Dict[str, Any]):
        for index in self._indexes:
            index.add_document(document)
```

```
def search(self, query_text: str, k: int = 1, k_rrf: int = 60):  
    # Отримати результати з усіх індексів  
    all_results = []  
    for idx, results in enumerate(all_results):  
        for rank, (doc, _) in enumerate(results):  
            # Відстежуємо рейтинги документів у різних індексах  
            # Застосувати формулу підрахунку балів RRF  
  
    # Повернути об'єднані та відсортовані результати
```

Ключовим висновком є те, що завдяки підтримці узгоджених API в різних реалізаціях пошуку ми можемо легко поєднувати їх без тісної взаємозалежності.

Тестування гібридного підходу

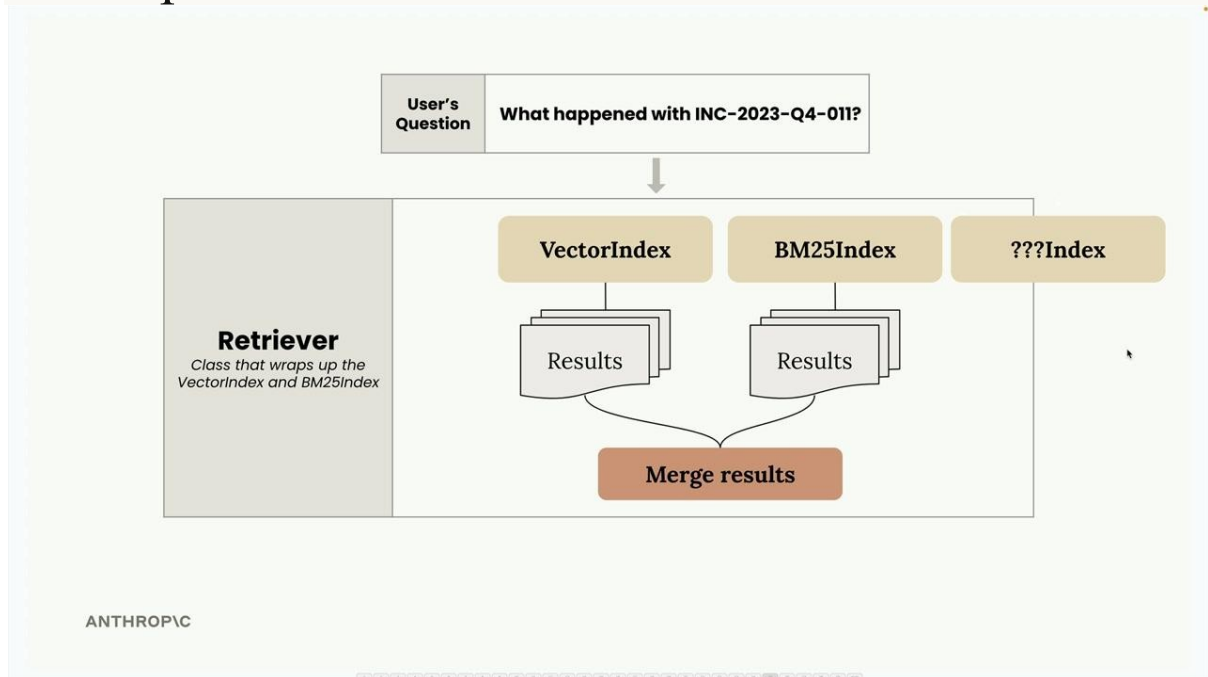
Пам'ятаєте нашу попередню проблему, коли пошук за запитом «що сталося з INC-2023-Q4-011?» повернув несподівані результати при використанні суто векторного підходу? На першому місці опинився інцидент з кібербезпеки (розділ 10), а на другому — фінансовий аналіз (розділ 3), замість більш релевантного розділу про розробку програмного забезпечення.

Завдяки нашому гібридному пошуковому механізму ми тепер отримуємо набагато кращі результати:

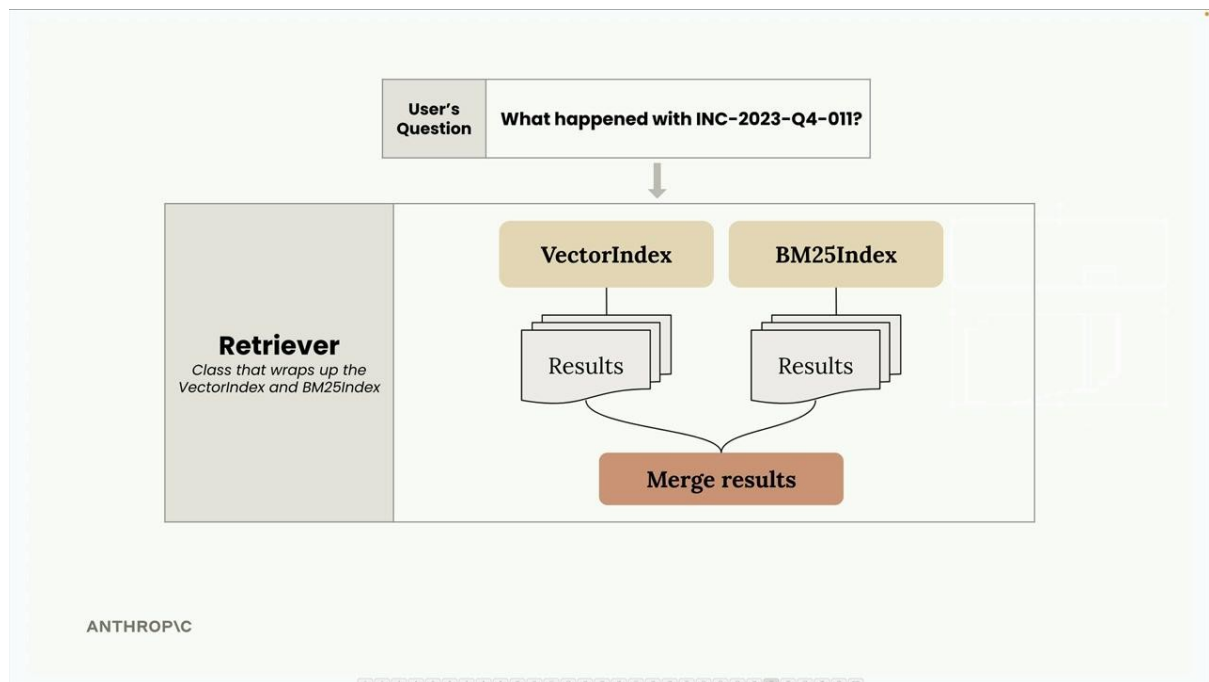
- Розділ 10: Аналіз кібербезпеки — Звіт про реагування на інцидент (найрелевантніший)
- Розділ 2: Розробка програмного забезпечення — Покращення стабільності проєкту «Фенікс» (другий за релевантністю)
- Розділ 5: Юридичні новини (третій за релевантністю)

Це демонструє, як поєднання семантичного та лексичного пошуку дозволяє подолати обмеження кожного з цих підходів, якщо застосовувати їх окремо.

Розширюваність



Перевага цієї архітектури полягає в її розширюваності. Оскільки всі індекси реалізують один і той самий протокол `SearchIndex` з методами `add_document()` та `search()`, ви можете легко додавати нові методи пошуку:



Хочете додати індекс на основі ключових слів? Пошук на основі графів?
Спеціалізований галузевий індекс? Просто реалізуйте той самий інтерфейс, і
Retriever
автоматично включить його в процес злиття.
Такий модульний підхід дозволяє кожній реалізації пошуку залишатися
цілеспрямованою та піддаватися тестуванню, водночас забезпечуючи чіткий
спосіб об'єднання їхніх сильних сторін у кінцевій
системі.